Lab 2: Cats vs Dogs

In this lab, you will train a convolutional neural network to classify an image into one of two classes: "cat" or "dog". The code for the neural networks you train will be written for you, and you are not (yet!) expected to understand all provided code. However, by the end of the lab, you should be able to:

1. Understand at a high level the training loop for a machine learning model.
2. Understand the distinction between training, validation, and test data.
3. The concepts of overfitting and underfitting.
4. Investigate how different hyperparameters, such as learning rate and batch size, affect the success of training.
5. Compare an ANN (aka Multi-Layer Perceptron) with a CNN.

What to submit

Submit a PDF file containing all your code, outputs, and write-up from parts 1-5. You can produce a PDF of your Google Colab file by going to **File > Print** and then save as PDF. The Colab instructions has more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Please use Google Colab to complete this assignment. If you want to use Jupyter Notebook, please complete the assignment and upload your Jupyter Notebook file to Google Colab for submission.

With Colab, you can export a PDF file using the menu option `File -> Print` and save as PDF file. **Adjust the scaling to ensure that the text is not cutoff at the margins.**

Colab Link

Include a link to your colab file here

Colab Link:
https://drive.google.com/file/d/1xrmGB757SezWNXDAGK6vx8yY1WK9Nb7j/view?usp=sharing

In [1]:
```
import numpy as np
import time
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
import torch.optim as optim
import torchvision
from torch.utils.data.sampler import SubsetRandomSampler
import torchvision.transforms as transforms
```

Part 0. Helper Functions

We will be making use of the following helper functions. You will be asked to look at and possibly modify some of these, but you are not expected to understand all of them.

You should look at the function names and read the docstrings. If you are curious, come back and explore the code *after* making some progress on the lab.

In [2]:
```
##############################################################################
# Data Loading

def get_relevant_indices(dataset, classes, target_classes):
    """ Return the indices for datapoints in the dataset that belongs to the
    desired target classes, a subset of all possible classes.

    Args:
        dataset: Dataset object
        classes: A list of strings denoting the name of each class
        target_classes: A list of strings denoting the name of desired classes
                        Should be a subset of the 'classes'
    Returns:
        indices: list of indices that have labels corresponding to one of the
                 target classes
    """
    indices = []
    for i in range(len(dataset)):
        # Check if the label is in the target classes
        label_index = dataset[i][1] # ex: 3
        label_class = classes[label_index] # ex: 'cat'
        if label_class in target_classes:
            indices.append(i)
    return indices

def get_data_loader(target_classes, batch_size):
    """ Loads images of cats and dogs, splits the data into training, validation
    and testing datasets. Returns data loaders for the three preprocessed datasets.

    Args:
        target_classes: A list of strings denoting the name of the desired
                        classes. Should be a subset of the argument 'classes'
        batch_size: A int representing the number of samples per batch

    Returns:
        train_loader: iterable training dataset organized according to batch size
        val_loader: iterable validation dataset organized according to batch size
        test_loader: iterable testing dataset organized according to batch size
        classes: A list of strings denoting the name of each class
    """
```

```
    classes = ('plane', 'car', 'bird', 'cat',
               'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
    ########################################################################
    # The output of torchvision datasets are PILImage images of range [0, 1].
    # We transform them to Tensors of normalized range [-1, 1].
    transform = transforms.Compose(
        [transforms.ToTensor(),
         transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
    # Load CIFAR10 training data
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                            download=True, transform=transform)
    # Get the list of indices to sample from
    relevant_indices = get_relevant_indices(trainset, classes, target_classes)

    # Split into train and validation
    np.random.seed(1000) # Fixed numpy random seed for reproducible shuffling
    np.random.shuffle(relevant_indices)
    split = int(len(relevant_indices) * 0.8) #split at 80%

    # split into training and validation indices
    relevant_train_indices, relevant_val_indices = relevant_indices[:split], releva
    train_sampler = SubsetRandomSampler(relevant_train_indices)
    train_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                               num_workers=1, sampler=train_sampler
    val_sampler = SubsetRandomSampler(relevant_val_indices)
    val_loader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                             num_workers=1, sampler=val_sampler)
    # Load CIFAR10 testing data
    testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                           download=True, transform=transform)
    # Get the list of indices to sample from
    relevant_test_indices = get_relevant_indices(testset, classes, target_classes)
    test_sampler = SubsetRandomSampler(relevant_test_indices)
    test_loader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                              num_workers=1, sampler=test_sampler)
    return train_loader, val_loader, test_loader, classes

########################################################################
# Training
def get_model_name(name, batch_size, learning_rate, epoch):
    """ Generate a name for the model consisting of all the hyperparameter values

    Args:
        config: Configuration object containing the hyperparameters
    Returns:
        path: A string with the hyperparameter name and value concatenated
    """
    path = "model_{0}_bs{1}_lr{2}_epoch{3}".format(name,
                                                   batch_size,
                                                   learning_rate,
                                                   epoch)
    return path

def normalize_label(labels):
    """
```

```
    Given a tensor containing 2 possible values, normalize this to 0/1

    Args:
        labels: a 1D tensor containing two possible scalar values
    Returns:
        A tensor normalize to 0/1 value
    """
    max_val = torch.max(labels)
    min_val = torch.min(labels)
    norm_labels = (labels - min_val)/(max_val - min_val)
    return norm_labels

def evaluate(net, loader, criterion):
    """ Evaluate the network on the validation set.

     Args:
         net: PyTorch neural network object
         loader: PyTorch data loader for the validation set
         criterion: The loss function
     Returns:
         err: A scalar for the avg classification error over the validation set
         loss: A scalar for the average loss function over the validation set
     """
    total_loss = 0.0
    total_err = 0.0
    total_epoch = 0
    for i, data in enumerate(loader, 0):
        inputs, labels = data
        labels = normalize_label(labels)  # Convert labels to 0/1
        outputs = net(inputs)
        loss = criterion(outputs, labels.float())
        corr = (outputs > 0.0).squeeze().long() != labels
        total_err += int(corr.sum())
        total_loss += loss.item()
        total_epoch += len(labels)
    err = float(total_err) / total_epoch
    loss = float(total_loss) / (i + 1)
    return err, loss

#############################################################################
# Training Curve
def plot_training_curve(path):
    """ Plots the training curve for a model run, given the csv files
    containing the train/validation error/loss.

    Args:
        path: The base path of the csv files produced during training
    """
    import matplotlib.pyplot as plt
    train_err = np.loadtxt("{}_train_err.csv".format(path))
    val_err = np.loadtxt("{}_val_err.csv".format(path))
    train_loss = np.loadtxt("{}_train_loss.csv".format(path))
    val_loss = np.loadtxt("{}_val_loss.csv".format(path))
    plt.title("Train vs Validation Error")
    n = len(train_err) # number of epochs
    plt.plot(range(1,n+1), train_err, label="Train")
```

```
    plt.plot(range(1,n+1), val_err, label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Error")
    plt.legend(loc='best')
    plt.show()
    plt.title("Train vs Validation Loss")
    plt.plot(range(1,n+1), train_loss, label="Train")
    plt.plot(range(1,n+1), val_loss, label="Validation")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend(loc='best')
    plt.show()
```

Part 1. Visualizing the Data [7 pt]

We will make use of some of the CIFAR-10 data set, which consists of colour images of size 32x32 pixels belonging to 10 categories. You can find out more about the dataset at https://www.cs.toronto.edu/~kriz/cifar.html

For this assignment, we will only be using the cat and dog categories. We have included code that automatically downloads the dataset the first time that the main script is run.

In [3]:
```
# This will download the CIFAR-10 dataset to a folder called "data"
# the first time you run this code.
train_loader, val_loader, test_loader, classes = get_data_loader(
    target_classes=["cat", "dog"],
    batch_size=1) # One image per batch
```

Part (a) -- 1 pt

Visualize some of the data by running the code below. Include the visualization in your writeup.

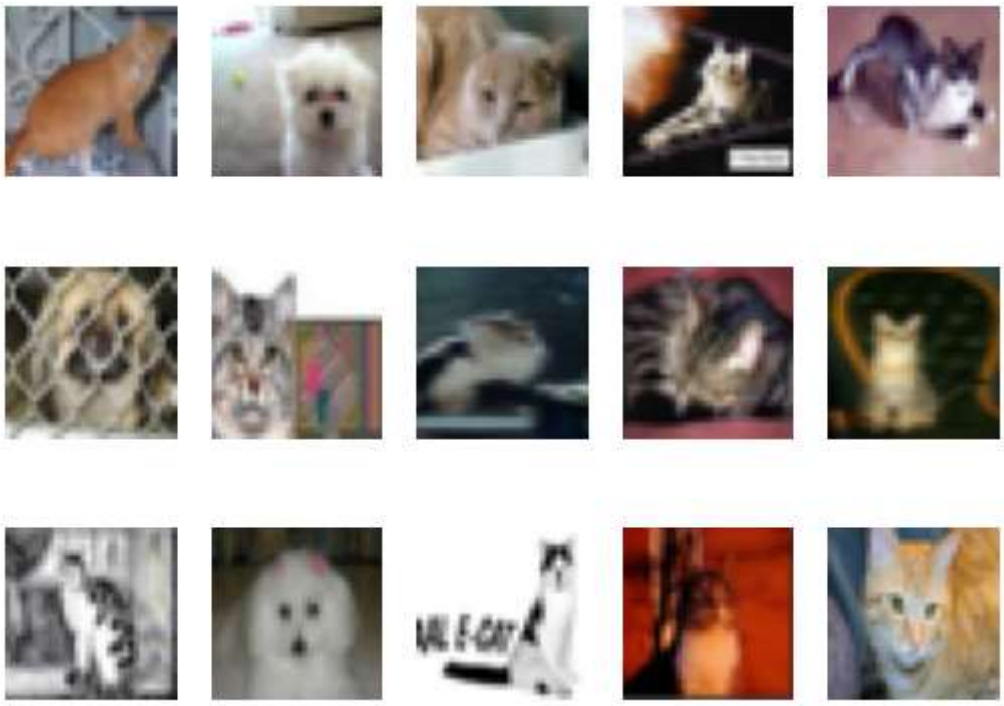
(You don't need to submit anything else.)

In [13]:
```
import matplotlib.pyplot as plt

k = 0
for images, labels in train_loader:
    # since batch_size = 1, there is only 1 image in `images`
    image = images[0]
    # place the colour channel at the end, instead of at the beginning
    img = np.transpose(image, [1,2,0])
    # normalize pixel intensity values to [0, 1]
    img = img / 2 + 0.5
    plt.subplot(3, 5, k+1)
    plt.axis('off')
    plt.imshow(img)

    k += 1
```

```
    if k > 14:
        break
```



Part (b) -- 3 pt

How many training examples do we have for the combined `cat` and `dog` classes? What about validation examples? What about test examples?

There are 8,000 images of cats and dogs for the training set, and 2,000 images for the validation set. There are also 2,000 test examples.

- CIFAR-10 has 5,000 training images per class, thus for cats and dogs, there are 10,000 in total
- The function "get_data_loader" splits the data into 80% training set and 20% validiation set.
- There are 1,000 test images per class

Part (c) -- 3pt

Why do we need a validation set when training our model? What happens if we judge the performance of our models using the training set loss/error instead of the validation set loss/error?

We need a validation set when training our model to measure how well the model performs on unseen data while training. It helps us choose the best model, hyperparamters, and stopping point. If we judge the performance based on just training loss/error, the model could look like it is improving just because it is memorizing the training examples. This is

called overfitting. Training error might be very low, while the validation error reamins high, meaning that the model does not do well when generalizing to new images.

Part 2. Training [15 pt]

We define two neural networks, a `LargeNet` and `SmallNet`. We'll be training the networks in this section.

You won't understand fully what these networks are doing until the next few classes, and that's okay. For this assignment, please focus on learning how to train networks, and how hyperparameters affect training.

In [4]:
```
class LargeNet(nn.Module):
    def __init__(self):
        super(LargeNet, self).__init__()
        self.name = "large"
        self.conv1 = nn.Conv2d(3, 5, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(5, 10, 5)
        self.fc1 = nn.Linear(10 * 5 * 5, 32)
        self.fc2 = nn.Linear(32, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 10 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        x = x.squeeze(1) # Flatten to [batch_size]
        return x
```

In [5]:
```
class SmallNet(nn.Module):
    def __init__(self):
        super(SmallNet, self).__init__()
        self.name = "small"
        self.conv = nn.Conv2d(3, 5, 3)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(5 * 7 * 7, 1)

    def forward(self, x):
        x = self.pool(F.relu(self.conv(x)))
        x = self.pool(x)
        x = x.view(-1, 5 * 7 * 7)
        x = self.fc(x)
        x = x.squeeze(1) # Flatten to [batch_size]
        return x
```

In [6]:
```
small_net = SmallNet()
large_net = LargeNet()
```

Part (a) -- 2pt

The methods `small_net.parameters()` and `large_net.parameters()` produces an iterator of all the trainable parameters of the network. These parameters are torch tensors containing many scalar values.

We haven't learned how how the parameters in these high-dimensional tensors will be used, but we should be able to count the number of parameters. Measuring the number of parameters in a network is one way of measuring the "size" of a network.

What is the total number of parameters in `small_net` and in `large_net`? (Hint: how many numbers are in each tensor?)

In [ ]:
```
for param in small_net.parameters():
    print(param.shape)
for param in large_net.parameters():
    print(param.shape)
```

```
torch.Size([5, 3, 3, 3])
torch.Size([5])
torch.Size([1, 245])
torch.Size([1])
torch.Size([5, 3, 5, 5])
torch.Size([5])
torch.Size([10, 5, 5, 5])
torch.Size([10])
torch.Size([32, 250])
torch.Size([32])
torch.Size([1, 32])
torch.Size([1])
```

Total parameters of small_net: 386 parameters

- Conv weights: 533*3=135
- Conv bias: 5
- Fully connected weights: 1*245=245
- Fully connected bias: 1
- Total: 135 + 5 + 245 + 1 = 386

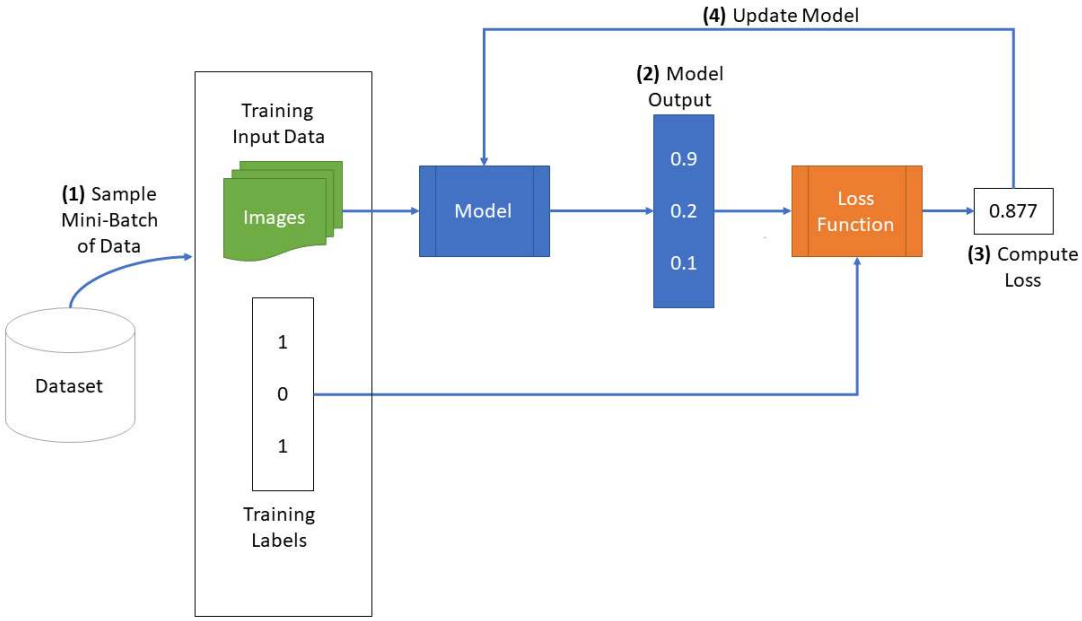
Total parameters for large_net: 9705 parameters

- conv1: 535*5 + 5 = 380
- conv2: 1055*5 + 10 = 1260
- fc1: 32*250 + 32 = 8032
- fc2: 1*32 + 1 = 33
- Total: 380 + 1260 + 8032 + 33 = 9705

The function train_net

The function `train_net` below takes an untrained neural network (like `small_net` and `large_net`) and several other parameters. You should be able to understand how this

function works. The figure below shows the high level training loop for a machine learning model:



In [7]:
```
def train_net(net, batch_size=64, learning_rate=0.01, num_epochs=30):
    ###############################################################################
    # Train a classifier on cats vs dogs
    target_classes = ["cat", "dog"]
    ###############################################################################
    # Fixed PyTorch random seed for reproducible result
    torch.manual_seed(1000)
    ###############################################################################
    # Obtain the PyTorch data loader objects to load batches of the datasets
    train_loader, val_loader, test_loader, classes = get_data_loader(
            target_classes, batch_size)
    ###############################################################################
    # Define the Loss function and optimizer
    # The loss function will be Binary Cross Entropy (BCE). In this case we
    # will use the BCEWithLogitsLoss which takes unnormalized output from
    # the neural network and scalar label.
    # Optimizer will be SGD with Momentum.
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.SGD(net.parameters(), lr=learning_rate, momentum=0.9)
    ###############################################################################
    # Set up some numpy arrays to store the training/test loss/erruracy
    train_err = np.zeros(num_epochs)
    train_loss = np.zeros(num_epochs)
    val_err = np.zeros(num_epochs)
    val_loss = np.zeros(num_epochs)
    ###############################################################################
    # Train the network
    # Loop over the data iterator and sample a new batch of training data
    # Get the output from the network, and optimize our loss function.
    start_time = time.time()
    for epoch in range(num_epochs):  # loop over the dataset multiple times
        total_train_loss = 0.0
```

```
        total_train_err = 0.0
        total_epoch = 0
        for i, data in enumerate(train_loader, 0):
            # Get the inputs
            inputs, labels = data
            labels = normalize_label(labels) # Convert labels to 0/1
            # Zero the parameter gradients
            optimizer.zero_grad()
            # Forward pass, backward pass, and optimize
            outputs = net(inputs)
            loss = criterion(outputs, labels.float())
            loss.backward()
            optimizer.step()
            # Calculate the statistics
            corr = (outputs > 0.0).squeeze().long() != labels
            total_train_err += int(corr.sum())
            total_train_loss += loss.item()
            total_epoch += len(labels)
        train_err[epoch] = float(total_train_err) / total_epoch
        train_loss[epoch] = float(total_train_loss) / (i+1)
        val_err[epoch], val_loss[epoch] = evaluate(net, val_loader, criterion)
        print(("Epoch {}: Train err: {}, Train loss: {} |"+
               "Validation err: {}, Validation loss: {}").format(
                   epoch + 1,
                   train_err[epoch],
                   train_loss[epoch],
                   val_err[epoch],
                   val_loss[epoch]))
        # Save the current model (checkpoint) to a file
        model_path = get_model_name(net.name, batch_size, learning_rate, epoch)
        torch.save(net.state_dict(), model_path)
    print('Finished Training')
    end_time = time.time()
    elapsed_time = end_time - start_time
    print("Total time elapsed: {:.2f} seconds".format(elapsed_time))
    # Write the train/test loss/err into CSV file for plotting later
    epochs = np.arange(1, num_epochs + 1)
    np.savetxt("{}_train_err.csv".format(model_path), train_err)
    np.savetxt("{}_train_loss.csv".format(model_path), train_loss)
    np.savetxt("{}_val_err.csv".format(model_path), val_err)
    np.savetxt("{}_val_loss.csv".format(model_path), val_loss)
```

Part (b) -- 1pt

The parameters to the function `train_net` are hyperparameters of our neural network. We made these hyperparameters easy to modify so that we can tune them later on.

What are the default values of the parameters `batch_size`, `learning_rate`, and `num_epochs`?

- batch_size = 64
- learning_rate = 0.01

- num_ephochs = 30

Part (c) -- 3 pt

What files are written to disk when we call `train_net` with `small_net`, and train for 5 epochs? Provide a list of all the files written to disk, and what information the files contain.

Model checkpoint files (One checkpoint is saved after easy epoch, each file contains the network weights after that epoch):

- model_small_bs64_lr0.01_epoch0
- model_small_bs64_lr0.01_epoch1
- model_small_bs64_lr0.01_epoch2
- model_small_bs64_lr0.01_epoch3
- model_small_bs64_lr0.01_epoch4

Training statistics file (after training, 4 csv files are created using the final checkpoint name, epoch4)

- model_small_bs64_lr0.01_epoch4_train_err_.csv : training classification error
- model_small_bs64_lr0.01_epoch4_train_loss_.csv : training loss
- model_small_bs64_lr0.01_epoch4_val_err_.csv : validation classification error
- model_small_bs64_lr0.01_epoch4_val_loss_.csv : validation loss

Part (d) -- 2pt

Train both `small_net` and `large_net` using the function `train_net` and its default parameters. The function will write many files to disk, including a model checkpoint (saved values of model weights) at the end of each epoch.

If you are using Google Colab, you will need to mount Google Drive so that the files generated by `train_net` gets saved. We will be using these files in part (d). (See the Google Colab tutorial for more information about this.)

Report the total time elapsed when training each network. Which network took longer to train? Why?

In [ ]:
```
# Since the function writes files to disk, you will need to mount
# your Google Drive. If you are working on the lab locally, you
# can comment out this code.

#from google.colab import drive
#drive.mount('/content/gdrive')
```

In [ ]:
```
# recording the total time taken to train the small network
small_net = SmallNet()
train_net(small_net, batch_size=64, learning_rate=0.01, num_epochs=30)
```

```
Epoch 1: Train err: 0.409625, Train loss: 0.6680866041183472 |Validation err: 0.38,
Validation loss: 0.6529379468411207
Epoch 2: Train err: 0.3495, Train loss: 0.636650381565094 |Validation err: 0.372, Va
lidation loss: 0.644148213788867
Epoch 3: Train err: 0.338, Train loss: 0.6187666630744935 |Validation err: 0.3375, V
alidation loss: 0.6197400242090225
Epoch 4: Train err: 0.3245, Train loss: 0.6040865285396576 |Validation err: 0.359, V
alidation loss: 0.627186669036746
Epoch 5: Train err: 0.316375, Train loss: 0.594923665523529 |Validation err: 0.333,
Validation loss: 0.6169400922954082
Epoch 6: Train err: 0.309375, Train loss: 0.5862700350284576 |Validation err: 0.328
5, Validation loss: 0.6125966534018517
Epoch 7: Train err: 0.3085, Train loss: 0.5833819851875305 |Validation err: 0.3295,
Validation loss: 0.608515608124435
Epoch 8: Train err: 0.29775, Train loss: 0.5781815807819366 |Validation err: 0.3175,
Validation loss: 0.6042346935719252
Epoch 9: Train err: 0.29725, Train loss: 0.5774542148113251 |Validation err: 0.319,
Validation loss: 0.6026235818862915
Epoch 10: Train err: 0.290375, Train loss: 0.5702568252086639 |Validation err: 0.308
5, Validation loss: 0.5977866565808654
Epoch 11: Train err: 0.294375, Train loss: 0.5712762188911438 |Validation err: 0.30
9, Validation loss: 0.5972736794501543
Epoch 12: Train err: 0.291375, Train loss: 0.5643501088619233 |Validation err: 0.31
2, Validation loss: 0.6050614193081856
Epoch 13: Train err: 0.287375, Train loss: 0.5664891231060029 |Validation err: 0.30
4, Validation loss: 0.5951512474566698
Epoch 14: Train err: 0.28675, Train loss: 0.560985102891922 |Validation err: 0.3065,
Validation loss: 0.6049496121704578
Epoch 15: Train err: 0.28225, Train loss: 0.5590886216163635 |Validation err: 0.302
5, Validation loss: 0.590254639275372
Epoch 16: Train err: 0.29075, Train loss: 0.5642498431205749 |Validation err: 0.314
5, Validation loss: 0.5996954003348947
Epoch 17: Train err: 0.2805, Train loss: 0.5579705429077149 |Validation err: 0.3005,
Validation loss: 0.5862716753035784
Epoch 18: Train err: 0.279375, Train loss: 0.5548499162197114 |Validation err: 0.304
5, Validation loss: 0.5863268095999956
Epoch 19: Train err: 0.28025, Train loss: 0.551603705406189 |Validation err: 0.314,
Validation loss: 0.6010742858052254
Epoch 20: Train err: 0.277625, Train loss: 0.5490770084857941 |Validation err: 0.289
5, Validation loss: 0.5883813267573714
Epoch 21: Train err: 0.279375, Train loss: 0.5530364484786987 |Validation err: 0.29
2, Validation loss: 0.5827296953648329
Epoch 22: Train err: 0.2785, Train loss: 0.5486431198120117 |Validation err: 0.302,
Validation loss: 0.6001685094088316
Epoch 23: Train err: 0.281125, Train loss: 0.5492630524635315 |Validation err: 0.295
5, Validation loss: 0.5861998815089464
Epoch 24: Train err: 0.276, Train loss: 0.5456731395721436 |Validation err: 0.294, V
alidation loss: 0.5815190887078643
Epoch 25: Train err: 0.274, Train loss: 0.542817405462265 |Validation err: 0.302, Va
lidation loss: 0.5875969249755144
Epoch 26: Train err: 0.268, Train loss: 0.5420180561542511 |Validation err: 0.301, V
alidation loss: 0.5809830715879798
Epoch 27: Train err: 0.273625, Train loss: 0.5418165931701661 |Validation err: 0.3,
Validation loss: 0.5881302198395133
Epoch 28: Train err: 0.27575, Train loss: 0.545127163887024 |Validation err: 0.3, Va
lidation loss: 0.5850601624697447
```

```
Epoch 29: Train err: 0.274375, Train loss: 0.5439188477993011 |Validation err: 0.30
4, Validation loss: 0.592289156280458
Epoch 30: Train err: 0.273125, Train loss: 0.5418467528820038 |Validation err: 0.30
3, Validation loss: 0.5827599205076694
Finished Training
Total time elapsed: 456.26 seconds
Total time taken to train the small network: 462.30 seconds
```

In [ ]:
```
#recording the total time taken to train the large network
large_net = LargeNet()
train_net(large_net, batch_size=64, learning_rate=0.01, num_epochs=30)
```

```
Epoch 1: Train err: 0.44475, Train loss: 0.6900203113555908 |Validation err: 0.4285,
Validation loss: 0.6807542983442545
Epoch 2: Train err: 0.419125, Train loss: 0.6781901936531067 |Validation err: 0.413,
Validation loss: 0.6741814073175192
Epoch 3: Train err: 0.39825, Train loss: 0.6658476033210754 |Validation err: 0.3895,
Validation loss: 0.6518005970865488
Epoch 4: Train err: 0.374, Train loss: 0.6491246662139892 |Validation err: 0.415, Va
lidation loss: 0.6677671857178211
Epoch 5: Train err: 0.355625, Train loss: 0.633103331565857 |Validation err: 0.355,
Validation loss: 0.6295729540288448
Epoch 6: Train err: 0.341375, Train loss: 0.6160089151859284 |Validation err: 0.339,
Validation loss: 0.616680096834898
Epoch 7: Train err: 0.324875, Train loss: 0.6004832179546357 |Validation err: 0.329
5, Validation loss: 0.6078574452549219
Epoch 8: Train err: 0.312375, Train loss: 0.5828952312469482 |Validation err: 0.324,
Validation loss: 0.5972189512103796
Epoch 9: Train err: 0.30725, Train loss: 0.5763978860378265 |Validation err: 0.3195,
Validation loss: 0.5960180722177029
Epoch 10: Train err: 0.29275, Train loss: 0.5610950827598572 |Validation err: 0.312,
Validation loss: 0.5912096621468663
Epoch 11: Train err: 0.28575, Train loss: 0.5485287392139435 |Validation err: 0.31,
Validation loss: 0.6070002932101488
Epoch 12: Train err: 0.277875, Train loss: 0.5412994689941406 |Validation err: 0.30
1, Validation loss: 0.5950419139117002
Epoch 13: Train err: 0.272, Train loss: 0.529847764492035 |Validation err: 0.308, Va
lidation loss: 0.5875048255547881
Epoch 14: Train err: 0.264, Train loss: 0.5180869660377503 |Validation err: 0.308, V
alidation loss: 0.5961919305846095
Epoch 15: Train err: 0.254625, Train loss: 0.5116696329116821 |Validation err: 0.297
5, Validation loss: 0.5886753043159842
Epoch 16: Train err: 0.253125, Train loss: 0.5054919605255127 |Validation err: 0.302
5, Validation loss: 0.5948434295132756
Epoch 17: Train err: 0.24975, Train loss: 0.4967941510677338 |Validation err: 0.301
5, Validation loss: 0.5864704167470336
Epoch 18: Train err: 0.233875, Train loss: 0.4819127686023712 |Validation err: 0.30
3, Validation loss: 0.6033055866137147
Epoch 19: Train err: 0.233625, Train loss: 0.47661114716529845 |Validation err: 0.3
2, Validation loss: 0.6075766989961267
Epoch 20: Train err: 0.22675, Train loss: 0.4655753197669983 |Validation err: 0.305
5, Validation loss: 0.6173305101692677
Epoch 21: Train err: 0.22275, Train loss: 0.45588080525398256 |Validation err: 0.290
5, Validation loss: 0.591132384724915
Epoch 22: Train err: 0.215375, Train loss: 0.4471959781646728 |Validation err: 0.295
5, Validation loss: 0.6126207308843732
Epoch 23: Train err: 0.208625, Train loss: 0.4410554223060608 |Validation err: 0.30
3, Validation loss: 0.6004117466509342
Epoch 24: Train err: 0.2045, Train loss: 0.4247907141447067 |Validation err: 0.3145,
Validation loss: 0.6618522200733423
Epoch 25: Train err: 0.194875, Train loss: 0.4110117611885071 |Validation err: 0.296
5, Validation loss: 0.6281136916950345
Epoch 26: Train err: 0.185875, Train loss: 0.3957132346630096 |Validation err: 0.29
5, Validation loss: 0.6556367529556155
Epoch 27: Train err: 0.172125, Train loss: 0.3771702340841293 |Validation err: 0.302
5, Validation loss: 0.6554299890995026
Epoch 28: Train err: 0.174, Train loss: 0.37409806180000305 |Validation err: 0.2965,
Validation loss: 0.6573651060461998
```

```
Epoch 29: Train err: 0.164625, Train loss: 0.3608104113340378 |Validation err: 0.308
5, Validation loss: 0.8224350176751614
Epoch 30: Train err: 0.152, Train loss: 0.342310009598732 |Validation err: 0.303, Va
lidation loss: 0.7170419273898005
Finished Training
Total time elapsed: 388.36 seconds
Total time taken to train the large network: 401.69 seconds
```

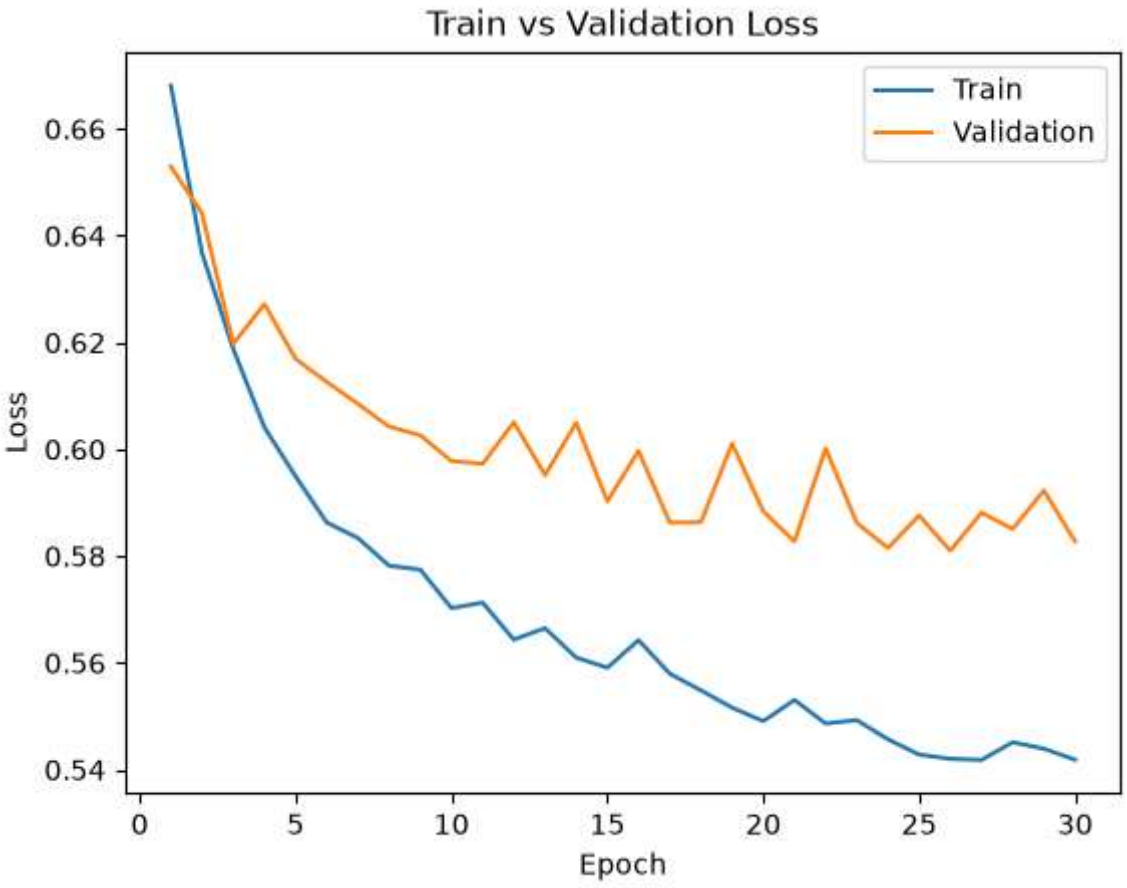
The total time elapsed to train the small network was 456.26 seconds, and the time it took to train the large network was 388.36. It took longer to train the smaller network. The measured time includes data loading and system overhead. The smaller network may have less efficiant operations for the hardware, whereas the larger network can use the computation more efficiently.

Part (e) - 2pt

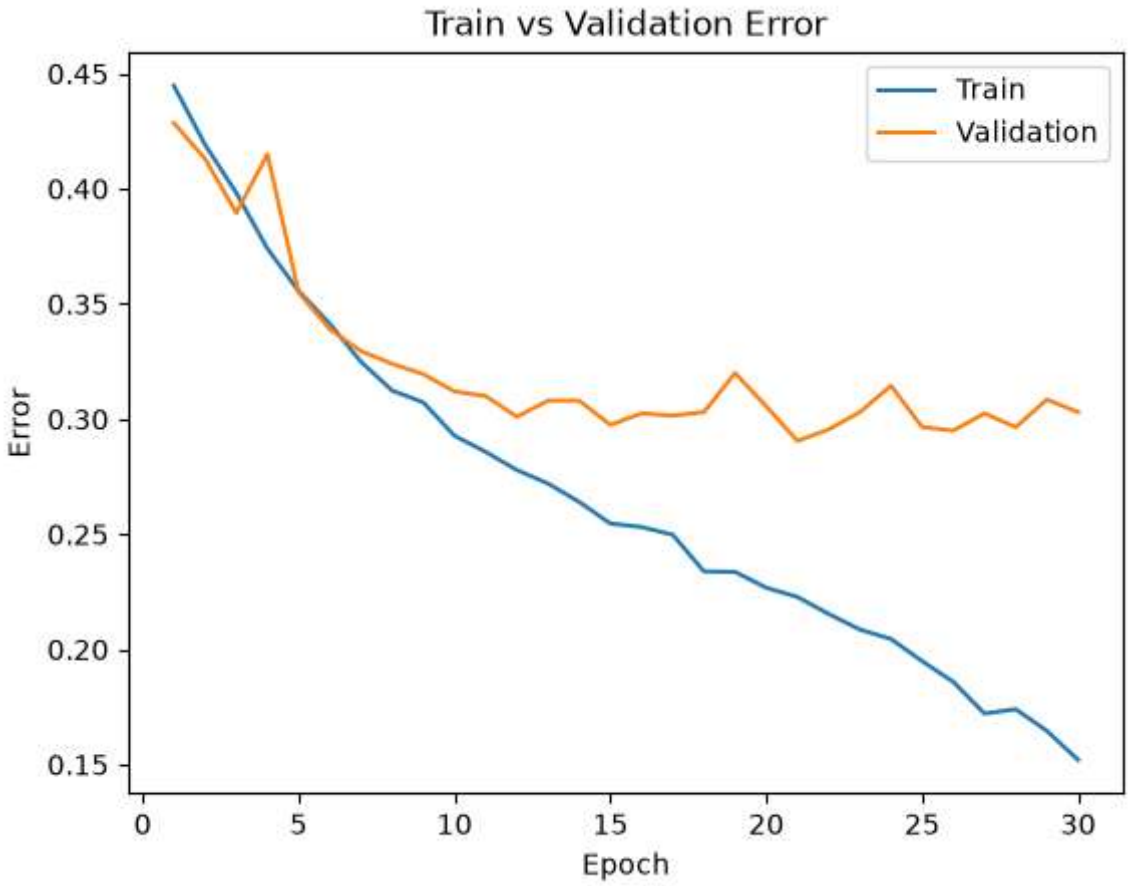
Use the function `plot_training_curve` to display the trajectory of the training/validation error and the training/validation loss. You will need to use the function `get_model_name` to generate the argument to the `plot_training_curve` function.

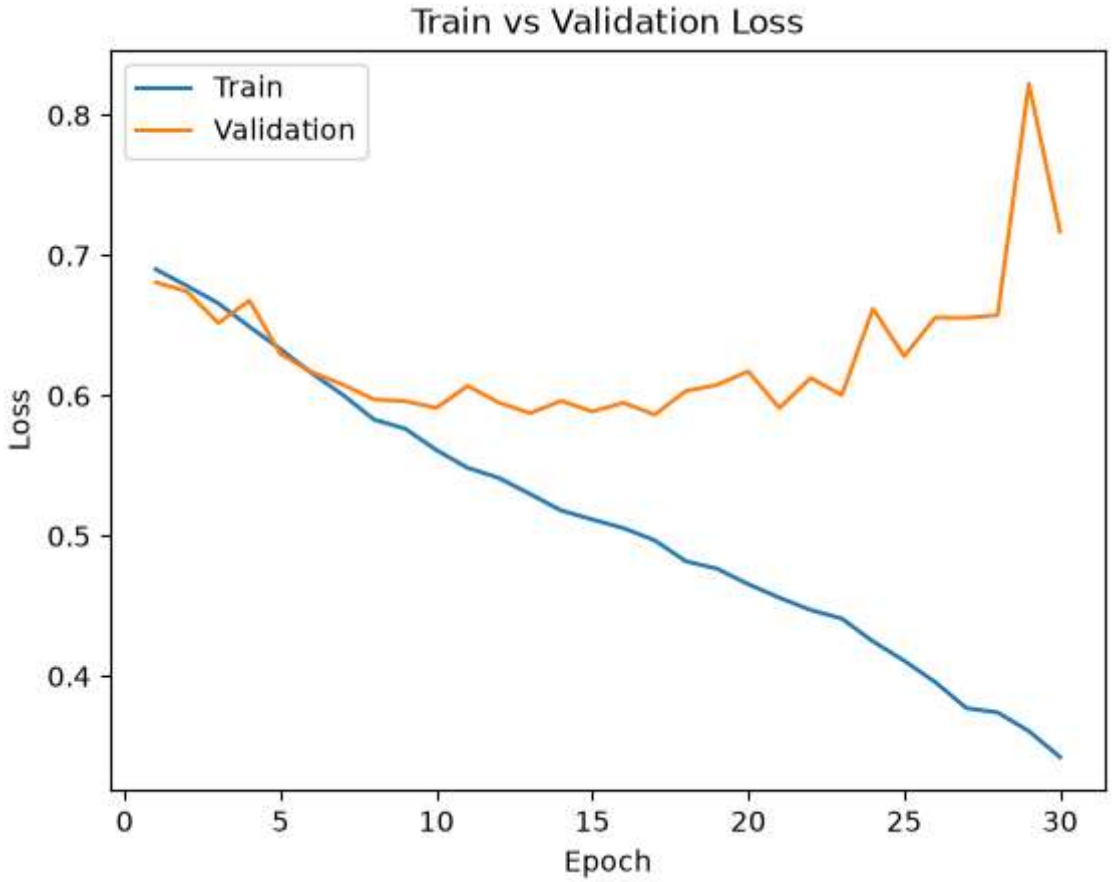
Do this for both the small network and the large network. Include both plots in your writeup.

In [ ]:
```
#model_path = get_model_name("small", batch_size=??, learning_rate=??, epoch=29)
# for small network
plot_training_curve(get_model_name("small", batch_size=64, learning_rate=0.01, epoc
```



In [ ]:
```
# for large network
plot_training_curve(get_model_name("large", batch_size=64, learning_rate=0.01, epoc
```


Train vs Validation Error
Train
Validation
Error
Epoch
0.45
0.40
0.35
0.30
0.25
0.20
0.15
0
5
10
15
20
25
30


Train vs Validation Loss
Train
Validation
Loss
Epoch
0.8
0.7
0.6
0.5
0.4
0
5
10
15
20
25
30

Part (f) - 5pt

Describe what you notice about the training curve. How do the curves differ for `small_net` and `large_net`? Identify any occurences of underfitting and overfitting.

For small_net, both the training and validation error decrease during the first few epochs. The training error continues to decrease gradually, while the validation error fluctuates around 0.30 and improves only slightly. The training and validation curves remain relatively close, so there is no strong evidence of overfitting. The relatively high training error suggests that small_net may be slightly underfitting because its limited capacity prevents it from learning the data completely.

For large_net, the training error and loss decrease consistently throughout training. However, after approximately epoch 8-10, the validation error and loss stop improving and begin to fluctuate or increase, while the training performance continues improving. This growing gap between the training and validation curves indicates overfitting. The best performance likely occurs around the point where the validation error or loss is lowest, before the curves diverge.

Overall, large_net fits the training data better than small_net, but it overfits more noticeably. small_net generalizes more consistently but has slightly weaker performance and may be mildly underfit.

Part 3. Optimization Parameters [12 pt]

For this section, we will work with `large_net` only.

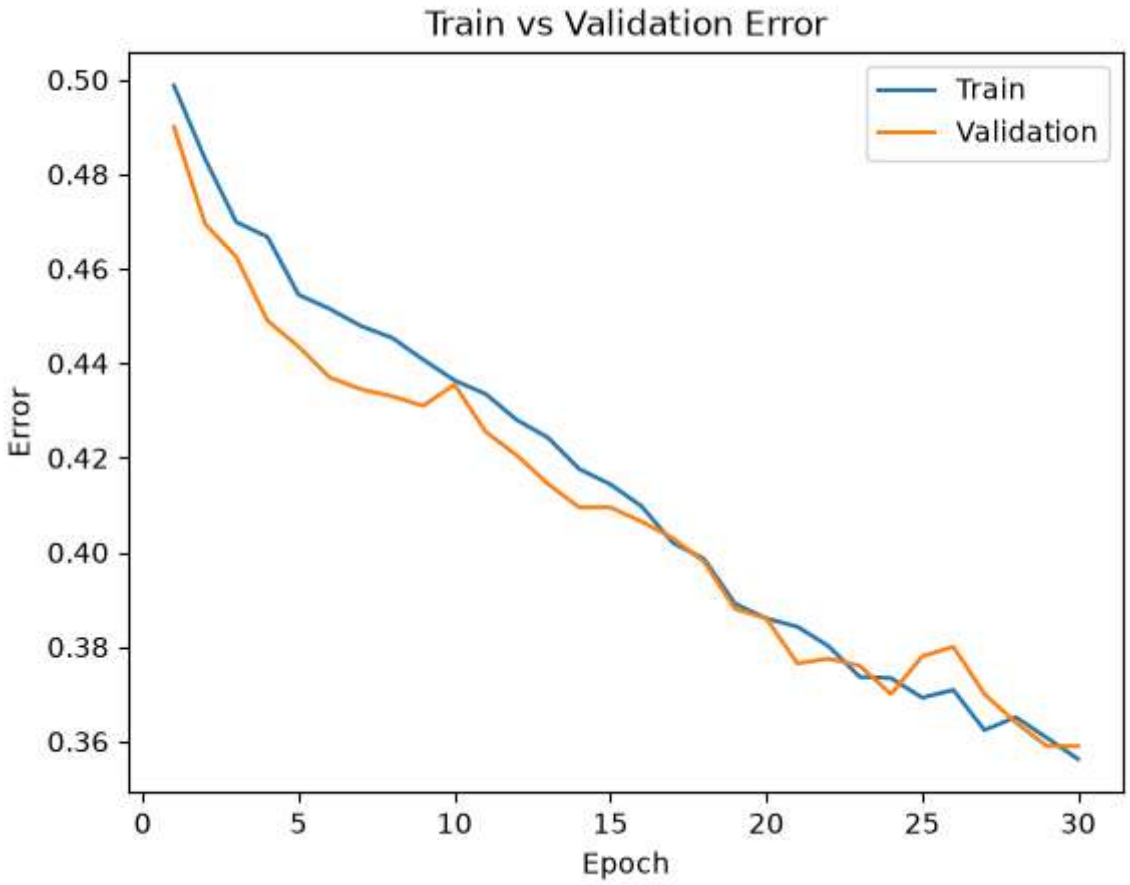
Part (a) - 3pt

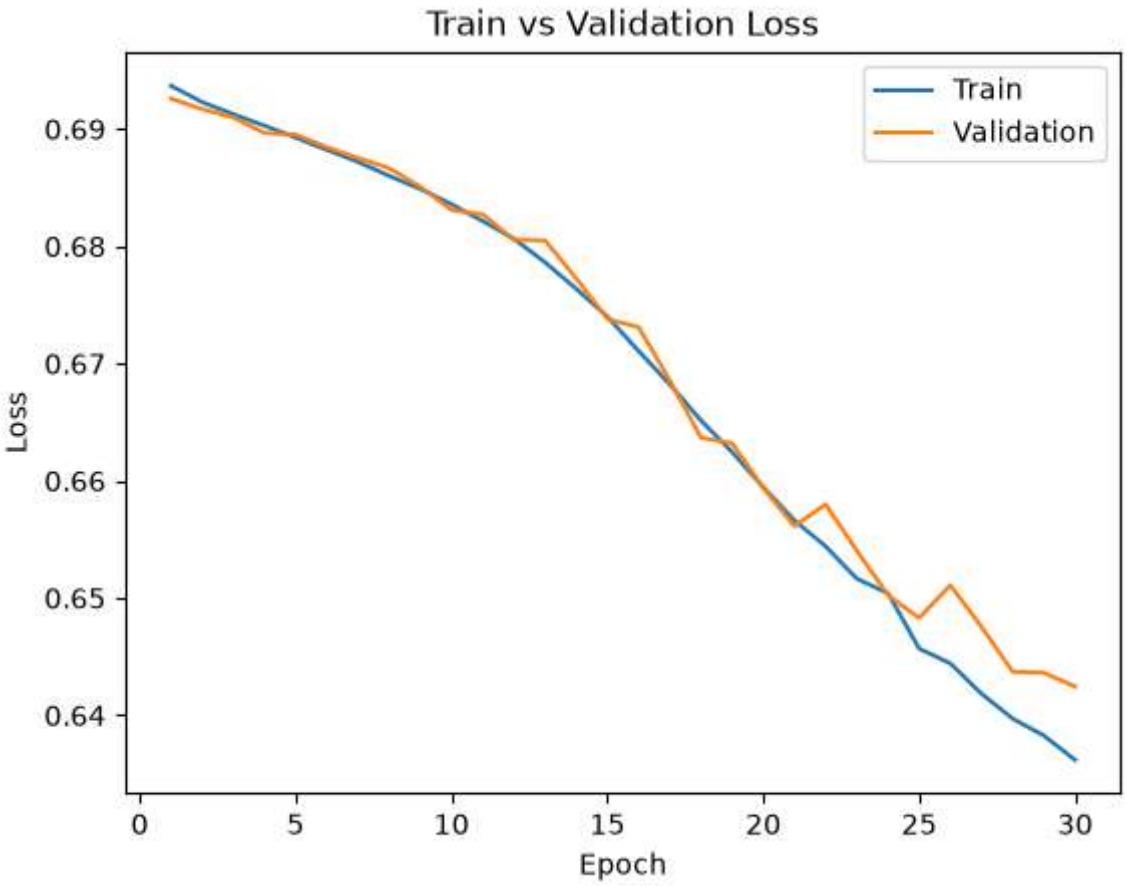
Train `large_net` with all default parameters, except set `learning_rate=0.001`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *lowering* the learning rate.

In [ ]:
```
# Note: When we re-construct the model, we start the training
# with *random weights*. If we omit this code, the values of
# the weights will still be the previously trained values.
large_net = LargeNet()
train_net(large_net, batch_size=64, learning_rate=0.001, num_epochs=30)
plot_training_curve(get_model_name("large", batch_size=64, learning_rate=0.001, epo
```

```
Epoch 1: Train err: 0.49875, Train loss: 0.6936983728408813 |Validation err: 0.49, V
alidation loss: 0.6925976295024157
Epoch 2: Train err: 0.48325, Train loss: 0.6923024954795838 |Validation err: 0.4695,
Validation loss: 0.6917003747075796
Epoch 3: Train err: 0.469875, Train loss: 0.6912673392295837 |Validation err: 0.462
5, Validation loss: 0.6909913923591375
Epoch 4: Train err: 0.46675, Train loss: 0.6902870445251464 |Validation err: 0.449,
Validation loss: 0.6896635591983795
Epoch 5: Train err: 0.4545, Train loss: 0.6892736902236939 |Validation err: 0.4435,
Validation loss: 0.6895225625485182
Epoch 6: Train err: 0.4515, Train loss: 0.6882340173721313 |Validation err: 0.437, V
alidation loss: 0.688451137393713
Epoch 7: Train err: 0.447875, Train loss: 0.6871999559402466 |Validation err: 0.434
5, Validation loss: 0.6875279098749161
Epoch 8: Train err: 0.445375, Train loss: 0.686003095626831 |Validation err: 0.433,
Validation loss: 0.6866629496216774
Epoch 9: Train err: 0.44075, Train loss: 0.6848909854888916 |Validation err: 0.431,
Validation loss: 0.6850748956203461
Epoch 10: Train err: 0.436375, Train loss: 0.6835821161270141 |Validation err: 0.435
5, Validation loss: 0.6830973010510206
Epoch 11: Train err: 0.4335, Train loss: 0.6821533489227295 |Validation err: 0.4255,
Validation loss: 0.6827144809067249
Epoch 12: Train err: 0.428, Train loss: 0.6806459498405456 |Validation err: 0.4205,
Validation loss: 0.6805648375302553
Epoch 13: Train err: 0.42425, Train loss: 0.6786349210739135 |Validation err: 0.414
5, Validation loss: 0.6805008705705404
Epoch 14: Train err: 0.417625, Train loss: 0.6763734579086303 |Validation err: 0.409
5, Validation loss: 0.6772713735699654
Epoch 15: Train err: 0.414375, Train loss: 0.6740030760765076 |Validation err: 0.409
5, Validation loss: 0.6737991292029619
Epoch 16: Train err: 0.40975, Train loss: 0.6710726013183593 |Validation err: 0.406
5, Validation loss: 0.6731040570884943
Epoch 17: Train err: 0.402, Train loss: 0.6682419419288635 |Validation err: 0.403, V
alidation loss: 0.66857117600739
Epoch 18: Train err: 0.398625, Train loss: 0.6651479921340943 |Validation err: 0.39
8, Validation loss: 0.663677217438817
Epoch 19: Train err: 0.389125, Train loss: 0.6624557828903198 |Validation err: 0.38
8, Validation loss: 0.6632092446088791
Epoch 20: Train err: 0.386, Train loss: 0.6594532227516174 |Validation err: 0.386, V
alidation loss: 0.6593825798481703
Epoch 21: Train err: 0.38425, Train loss: 0.656618486404419 |Validation err: 0.3765,
Validation loss: 0.656130438670516
Epoch 22: Train err: 0.380125, Train loss: 0.6544087715148926 |Validation err: 0.377
5, Validation loss: 0.6579940505325794
Epoch 23: Train err: 0.373625, Train loss: 0.6516457958221435 |Validation err: 0.37
6, Validation loss: 0.6540427636355162
Epoch 24: Train err: 0.373375, Train loss: 0.6504061770439148 |Validation err: 0.37,
Validation loss: 0.6502421870827675
Epoch 25: Train err: 0.36925, Train loss: 0.6456841711997986 |Validation err: 0.378,
Validation loss: 0.6482992302626371
Epoch 26: Train err: 0.370875, Train loss: 0.6444227867126465 |Validation err: 0.38,
Validation loss: 0.6510878149420023
Epoch 27: Train err: 0.362375, Train loss: 0.6418361988067627 |Validation err: 0.37,
Validation loss: 0.6475344374775887
Epoch 28: Train err: 0.365125, Train loss: 0.6397103514671326 |Validation err: 0.36
4, Validation loss: 0.6437127161771059
```

```
Epoch 29: Train err: 0.36075, Train loss: 0.6382656235694886 |Validation err: 0.359,
Validation loss: 0.6436217427253723
Epoch 30: Train err: 0.35625, Train loss: 0.6361900897026062 |Validation err: 0.359,
Validation loss: 0.6424481701105833
Finished Training
Total time elapsed: 352.37 seconds
```





Lowering the learning rate from 0.01 to 0.001 decreased the training time from 388.36 seconds to 352.37 seconds. With learning_rate=0.001, both the training and validation error decrease gradually throughout the 30 epochs. The training and validation curves remain close together, and the validation loss does not diverge significantly from the training loss. Therefore, there is little evidence of overfitting. The lower learning rate makes smaller weight updates, so the model learns more slowly and may require more epochs to reach a low error. After 30 epochs, the error is still relatively high, suggesting that the model may be underfitting or has not had enough time to converge.

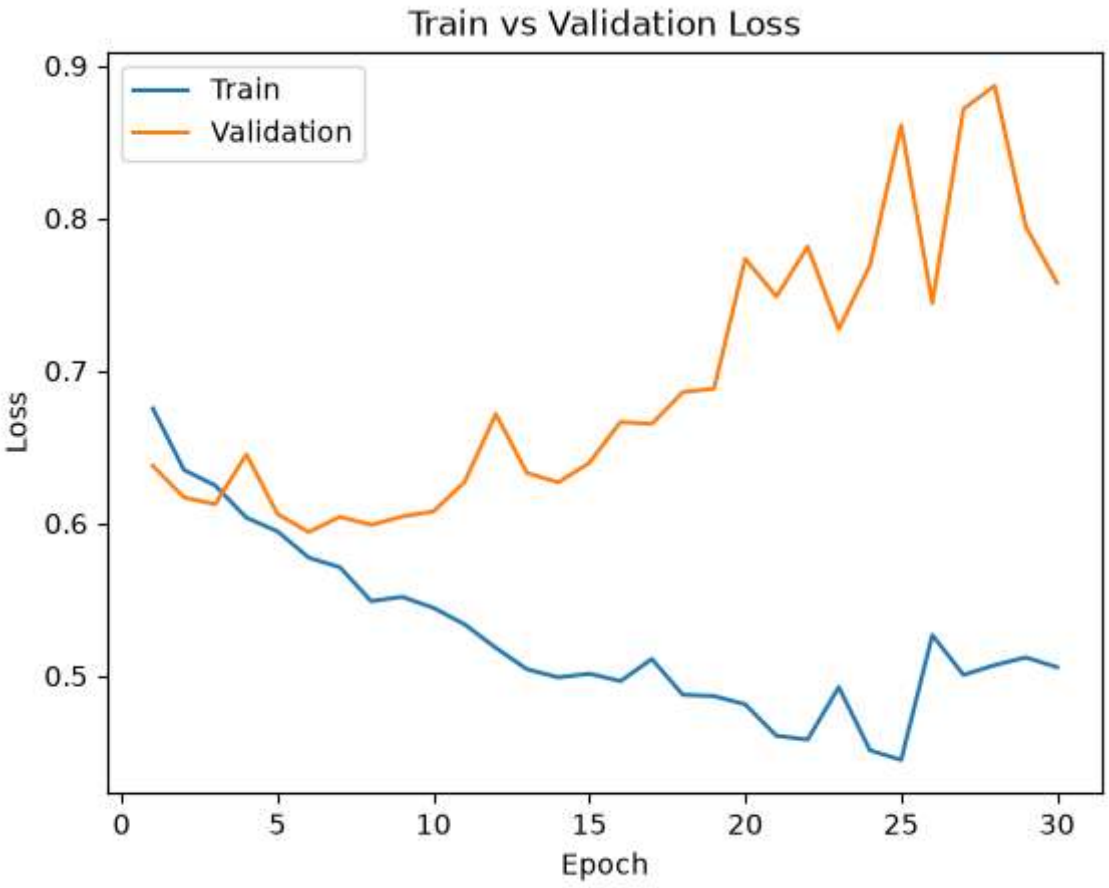
Part (b) - 3pt

Train `large_net` with all default parameters, except set `learning_rate=0.1`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *increasing* the learning rate.

In [ ]:
```
large_net = LargeNet()
train_net(large_net, batch_size=64, learning_rate=0.1, num_epochs=30)
plot_training_curve(get_model_name("large", batch_size=64, learning_rate=0.1, epoch
```

```
Epoch 1: Train err: 0.429875, Train loss: 0.6752145924568176 |Validation err: 0.378,
Validation loss: 0.6376289371401072
Epoch 2: Train err: 0.358625, Train loss: 0.6348985934257507 |Validation err: 0.353,
Validation loss: 0.6168579142540693
Epoch 3: Train err: 0.35125, Train loss: 0.624727258682251 |Validation err: 0.338, V
alidation loss: 0.6124672368168831
Epoch 4: Train err: 0.331, Train loss: 0.60360107421875 |Validation err: 0.351, Vali
dation loss: 0.6450391616672277
Epoch 5: Train err: 0.319, Train loss: 0.594632473230362 |Validation err: 0.34, Vali
dation loss: 0.6060001626610756
Epoch 6: Train err: 0.31025, Train loss: 0.5773915741443634 |Validation err: 0.321,
Validation loss: 0.5941676832735538
Epoch 7: Train err: 0.301625, Train loss: 0.5711239471435546 |Validation err: 0.317,
Validation loss: 0.6042191609740257
Epoch 8: Train err: 0.2875, Train loss: 0.5489215927124024 |Validation err: 0.34, Va
lidation loss: 0.598825397901237
Epoch 9: Train err: 0.28975, Train loss: 0.5516727933883667 |Validation err: 0.3215,
Validation loss: 0.6044350583106279
Epoch 10: Train err: 0.27775, Train loss: 0.5445137403011322 |Validation err: 0.308,
Validation loss: 0.607685943134129
Epoch 11: Train err: 0.2705, Train loss: 0.5336579625606537 |Validation err: 0.305,
Validation loss: 0.6270539034157991
Epoch 12: Train err: 0.259875, Train loss: 0.5182961461544037 |Validation err: 0.32
3, Validation loss: 0.6715641496703029
Epoch 13: Train err: 0.25225, Train loss: 0.5042386982440948 |Validation err: 0.305,
Validation loss: 0.6328667476773262
Epoch 14: Train err: 0.245375, Train loss: 0.4989571797847748 |Validation err: 0.317
5, Validation loss: 0.6267419597133994
Epoch 15: Train err: 0.248125, Train loss: 0.5013063044548035 |Validation err: 0.30
7, Validation loss: 0.6396154910326004
Epoch 16: Train err: 0.246875, Train loss: 0.49650333666801455 |Validation err: 0.31
85, Validation loss: 0.6663552429527044
Epoch 17: Train err: 0.24825, Train loss: 0.5109712693691254 |Validation err: 0.347,
Validation loss: 0.665094768628478
Epoch 18: Train err: 0.23625, Train loss: 0.48750538754463196 |Validation err: 0.329
5, Validation loss: 0.6859440039843321
Epoch 19: Train err: 0.238, Train loss: 0.48666382002830505 |Validation err: 0.3405,
Validation loss: 0.6882260181009769
Epoch 20: Train err: 0.2305, Train loss: 0.48126319122314454 |Validation err: 0.319
5, Validation loss: 0.7733230851590633
Epoch 21: Train err: 0.227625, Train loss: 0.4606091723442078 |Validation err: 0.31
5, Validation loss: 0.7486248575150967
Epoch 22: Train err: 0.221125, Train loss: 0.45813710713386535 |Validation err: 0.32
35, Validation loss: 0.7813193816691637
Epoch 23: Train err: 0.23525, Train loss: 0.4925526168346405 |Validation err: 0.312,
Validation loss: 0.7272448539733887
Epoch 24: Train err: 0.20925, Train loss: 0.4510812437534332 |Validation err: 0.310
5, Validation loss: 0.7691468643024564
Epoch 25: Train err: 0.211, Train loss: 0.4448058929443359 |Validation err: 0.344, V
alidation loss: 0.860934317111969
Epoch 26: Train err: 0.251, Train loss: 0.5266589555740356 |Validation err: 0.395, V
alidation loss: 0.7443090416491032
Epoch 27: Train err: 0.239375, Train loss: 0.5005191400051117 |Validation err: 0.37
6, Validation loss: 0.8716014716774225
Epoch 28: Train err: 0.241375, Train loss: 0.5069779899120331 |Validation err: 0.35
9, Validation loss: 0.8868240565061569
```

```
Epoch 29: Train err: 0.249125, Train loss: 0.51202321767807 |Validation err: 0.329,
Validation loss: 0.7941862884908915
Epoch 30: Train err: 0.239, Train loss: 0.505581267118454 |Validation err: 0.349, Va
lidation loss: 0.7576891873031855
Finished Training
Total time elapsed: 355.78 seconds
```



Increasing the learning rate to 0.1 decreased the training time to 355.78 seconds. However, graphs show that the training error and training loss generally decrease, meaning that the model continues improving on the training data. However, the validation error and validation loss become unstable and increase significantly after the early epochs. This creates a large gap between the training and validation curves and indicates strong overfitting or unstable updates.

A learning rate of 0.1 causes much larger weight updates. These updates can overshoot good solutions, causing the validation performance to fluctuate and worsen. The best validation performance occurs relatively early, before the validation curves begin to increase. Overall, 0.1 is too large for this model: although training performance improves, generalization becomes much worse compared with the default learning rate of 0.01.

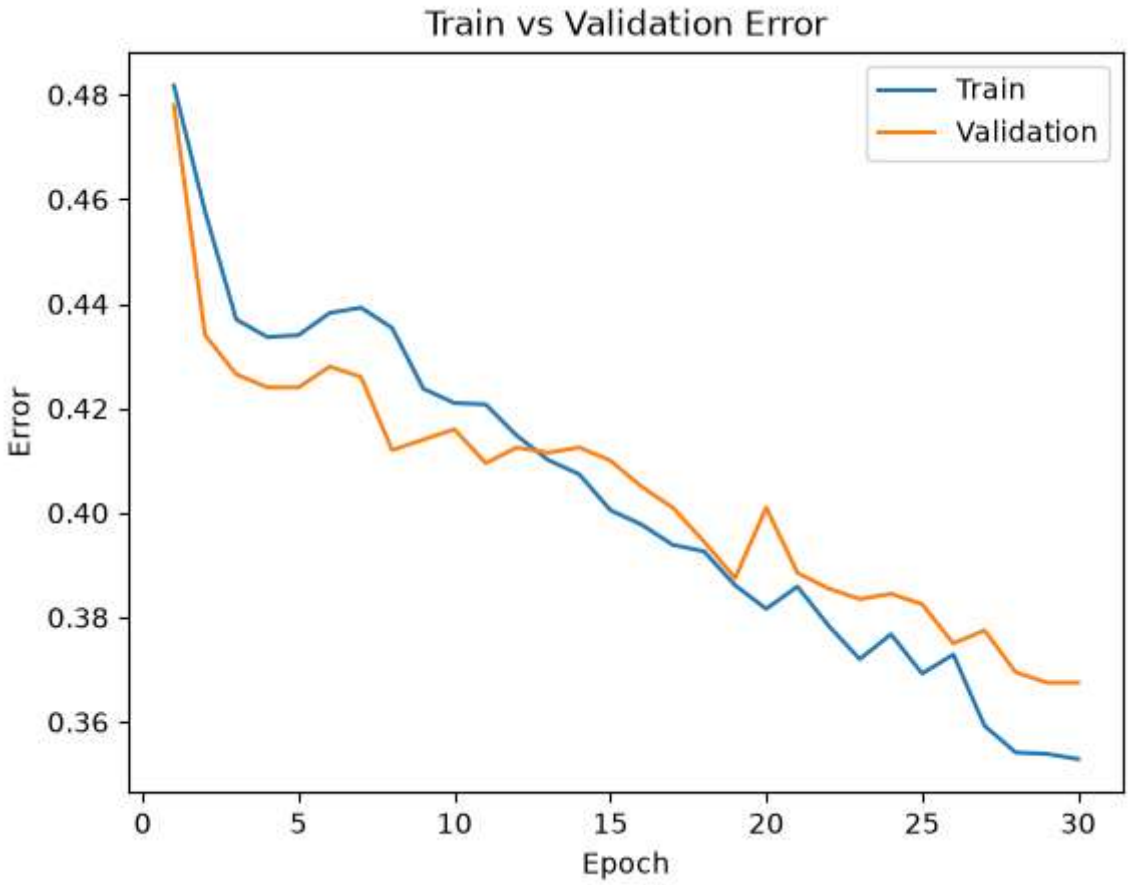
Part (c) - 3pt

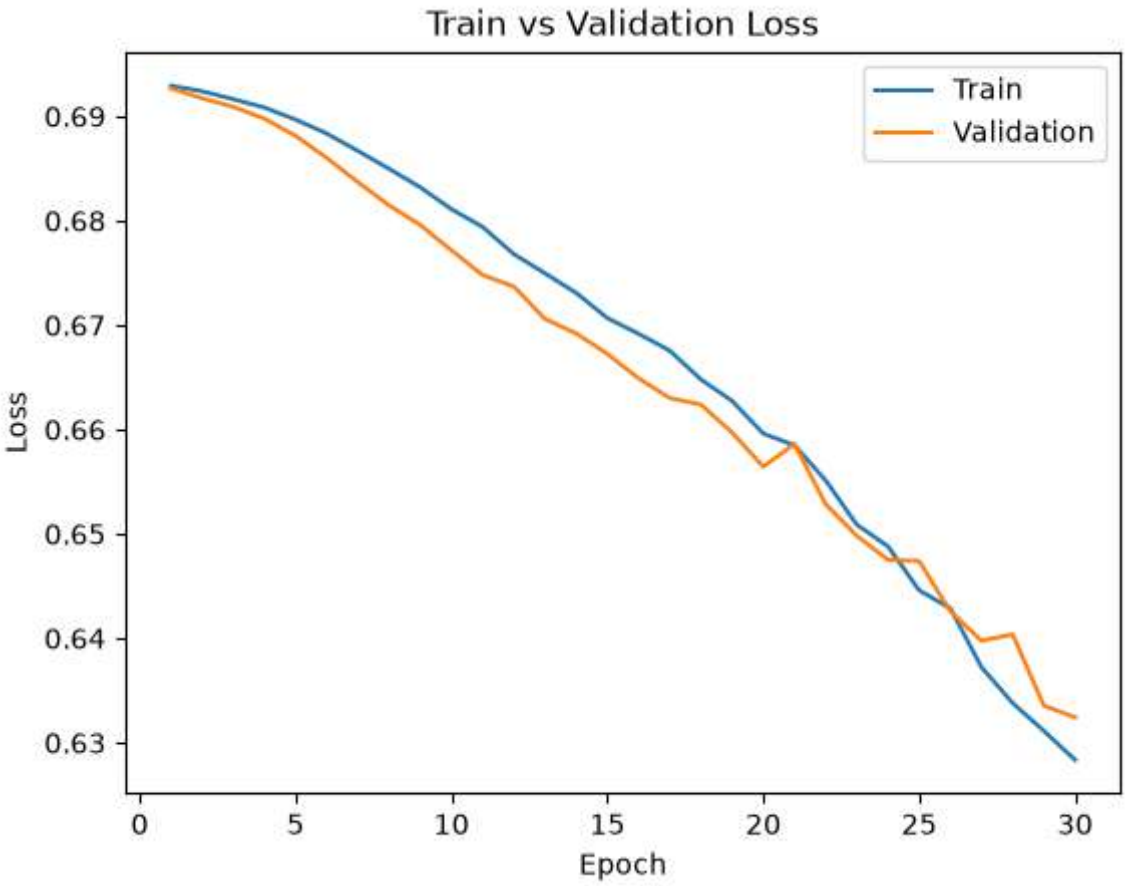
Train `large_net` with all default parameters, including with `learning_rate=0.01`. Now, set `batch_size=512`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *increasing* the batch size.

In [ ]:
```
large_net = LargeNet()
train_net(large_net, batch_size=512, learning_rate=0.01, num_epochs=30)
plot_training_curve(get_model_name("large", batch_size=512, learning_rate=0.01, epo
```

```
Epoch 1: Train err: 0.48175, Train loss: 0.6929379515349865 |Validation err: 0.478,
Validation loss: 0.6926824003458023
Epoch 2: Train err: 0.457625, Train loss: 0.6924103982746601 |Validation err: 0.434,
Validation loss: 0.6917425245046616
Epoch 3: Train err: 0.437, Train loss: 0.691650040447712 |Validation err: 0.4265, Va
lidation loss: 0.6909129321575165
Epoch 4: Train err: 0.433625, Train loss: 0.6908450089395046 |Validation err: 0.424,
Validation loss: 0.6897870749235153
Epoch 5: Train err: 0.434, Train loss: 0.6896936297416687 |Validation err: 0.424, Va
lidation loss: 0.6881358474493027
Epoch 6: Train err: 0.43825, Train loss: 0.6883534491062164 |Validation err: 0.428,
Validation loss: 0.6860133111476898
Epoch 7: Train err: 0.43925, Train loss: 0.6866872385144234 |Validation err: 0.426,
Validation loss: 0.6836972683668137
Epoch 8: Train err: 0.435375, Train loss: 0.6849771030247211 |Validation err: 0.412,
Validation loss: 0.6814650744199753
Epoch 9: Train err: 0.42375, Train loss: 0.6832010485231876 |Validation err: 0.414,
Validation loss: 0.6795920878648758
Epoch 10: Train err: 0.421, Train loss: 0.6811087094247341 |Validation err: 0.416, V
alidation loss: 0.6771562248468399
Epoch 11: Train err: 0.42075, Train loss: 0.679402619600296 |Validation err: 0.4095,
Validation loss: 0.6748088747262955
Epoch 12: Train err: 0.41475, Train loss: 0.6768024861812592 |Validation err: 0.412
5, Validation loss: 0.6736888438463211
Epoch 13: Train err: 0.410125, Train loss: 0.6749658435583115 |Validation err: 0.411
5, Validation loss: 0.6705993711948395
Epoch 14: Train err: 0.407375, Train loss: 0.6730865277349949 |Validation err: 0.412
5, Validation loss: 0.6692004799842834
Epoch 15: Train err: 0.4005, Train loss: 0.6706791669130325 |Validation err: 0.41, V
alidation loss: 0.6672518104314804
Epoch 16: Train err: 0.39775, Train loss: 0.6691761314868927 |Validation err: 0.405,
Validation loss: 0.6649051755666733
Epoch 17: Train err: 0.393875, Train loss: 0.667567253112793 |Validation err: 0.401,
Validation loss: 0.663018062710762
Epoch 18: Train err: 0.392625, Train loss: 0.664799626916647 |Validation err: 0.394
5, Validation loss: 0.6623904407024384
Epoch 19: Train err: 0.386125, Train loss: 0.6627435423433781 |Validation err: 0.387
5, Validation loss: 0.6597158461809158
Epoch 20: Train err: 0.381625, Train loss: 0.6596169136464596 |Validation err: 0.40
1, Validation loss: 0.6564357429742813
Epoch 21: Train err: 0.385875, Train loss: 0.6584909670054913 |Validation err: 0.388
5, Validation loss: 0.658623144030571
Epoch 22: Train err: 0.378375, Train loss: 0.6551269926130772 |Validation err: 0.385
5, Validation loss: 0.6528426855802536
Epoch 23: Train err: 0.372, Train loss: 0.6508895605802536 |Validation err: 0.3835,
Validation loss: 0.6498248279094696
Epoch 24: Train err: 0.37675, Train loss: 0.6488079577684402 |Validation err: 0.384
5, Validation loss: 0.6475121378898621
Epoch 25: Train err: 0.36925, Train loss: 0.6445969454944134 |Validation err: 0.382
5, Validation loss: 0.647397592663765
Epoch 26: Train err: 0.372875, Train loss: 0.6428787261247635 |Validation err: 0.37
5, Validation loss: 0.6425987482070923
Epoch 27: Train err: 0.35925, Train loss: 0.6372324824333191 |Validation err: 0.377
5, Validation loss: 0.639768660068512
Epoch 28: Train err: 0.354125, Train loss: 0.6338000074028969 |Validation err: 0.369
5, Validation loss: 0.6403798013925552
```

```
Epoch 29: Train err: 0.353875, Train loss: 0.6311283856630325 |Validation err: 0.367
5, Validation loss: 0.6335539370775223
Epoch 30: Train err: 0.352875, Train loss: 0.6283532157540321 |Validation err: 0.367
5, Validation loss: 0.6324330121278763
Finished Training
Total time elapsed: 342.19 seconds
```





Increasing the batch size to 512 decreased the training time to 342.19 second. This is expected since each epoch uses fewer batches. With 8,000 training examples, a batch size of 64 requires approximately 125 batches per epoch, while a batch size of 512 requires only approximately 16 batches per epoch. Therefore, the optimizer performs fewer updates during each epoch.

The training and validation error both decrease over the 30 epochs, although the validation error fluctuates slightly. The training and validation loss also decrease together and remain relatively close. This suggests that the model is learning steadily, with no strong evidence of overfitting in this run.

Increasing the batch size produces more stable, less noisy gradient estimates and reduces the number of parameter updates, making training faster. However, the model may learn less flexibly because it receives fewer updates per epoch. In this experiment, the final validation error is approximately 37%.

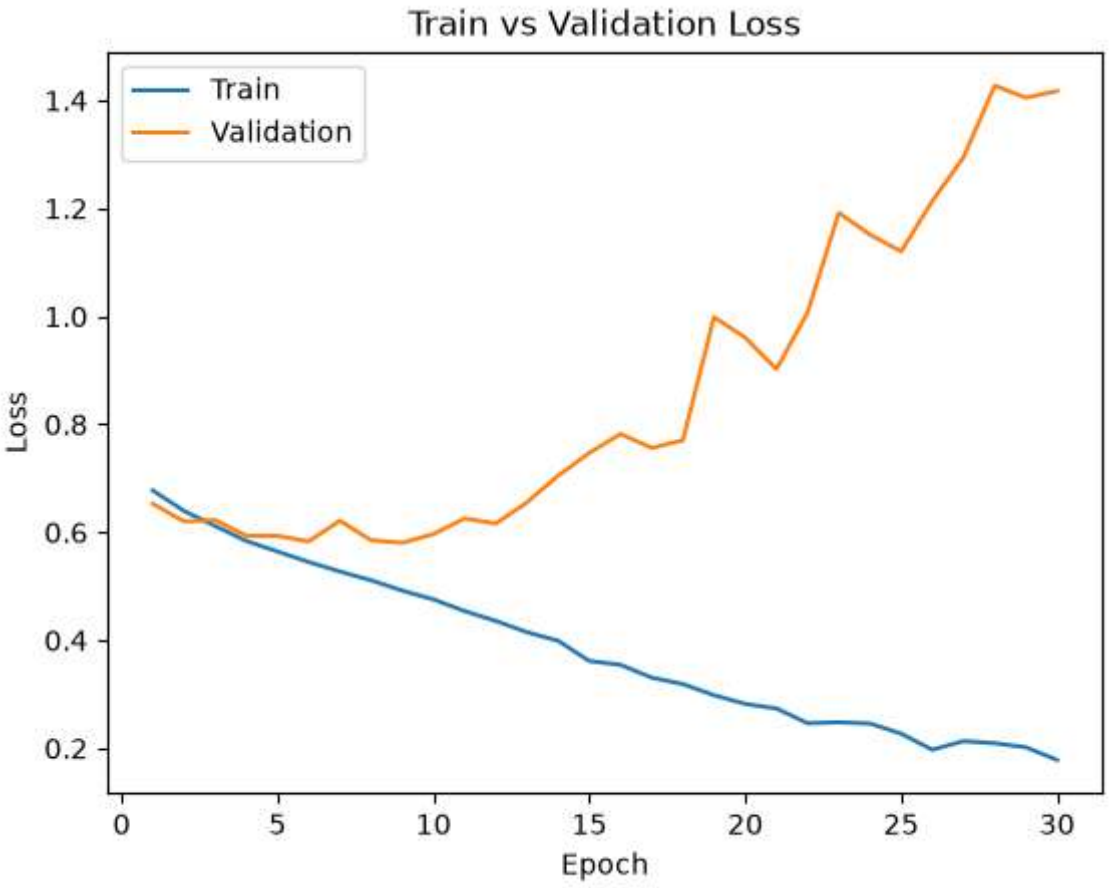
Part (d) - 3pt

Train `large_net` with all default parameters, including with `learning_rate=0.01`. Now, set `batch_size=16`. Does the model take longer/shorter to train? Plot the training curve. Describe the effect of *decreasing* the batch size.

In [ ]:
```
large_net = LargeNet()
train_net(large_net, batch_size=16, learning_rate=0.01, num_epochs=30)
plot_training_curve(get_model_name("large", batch_size=16, learning_rate=0.01, epoc
```

```
Epoch 1: Train err: 0.432625, Train loss: 0.6776098030805587 |Validation err: 0.374
5, Validation loss: 0.6529334607124329
Epoch 2: Train err: 0.3665, Train loss: 0.6396277079582214 |Validation err: 0.352, V
alidation loss: 0.6202217926979064
Epoch 3: Train err: 0.340375, Train loss: 0.611763548374176 |Validation err: 0.3425,
Validation loss: 0.6225978240966797
Epoch 4: Train err: 0.310375, Train loss: 0.583930772960186 |Validation err: 0.3335,
Validation loss: 0.5936324608325958
Epoch 5: Train err: 0.29975, Train loss: 0.5643941404819488 |Validation err: 0.3265,
Validation loss: 0.5934103174209595
Epoch 6: Train err: 0.281375, Train loss: 0.5449857948422432 |Validation err: 0.303
5, Validation loss: 0.5832211289405823
Epoch 7: Train err: 0.269375, Train loss: 0.5272681354284287 |Validation err: 0.319,
Validation loss: 0.621401073217392
Epoch 8: Train err: 0.263375, Train loss: 0.5111728370785713 |Validation err: 0.315,
Validation loss: 0.5854083404541016
Epoch 9: Train err: 0.2475, Train loss: 0.4920583007633686 |Validation err: 0.2945,
Validation loss: 0.5804896264076232
Epoch 10: Train err: 0.235875, Train loss: 0.47570361009240153 |Validation err: 0.29
45, Validation loss: 0.5968189946413041
Epoch 11: Train err: 0.223625, Train loss: 0.45409447586536406 |Validation err: 0.32
1, Validation loss: 0.6258061130046845
Epoch 12: Train err: 0.206, Train loss: 0.43577824819087985 |Validation err: 0.2995,
Validation loss: 0.6162136602401733
Epoch 13: Train err: 0.1955, Train loss: 0.41500603017210963 |Validation err: 0.31,
Validation loss: 0.6554790686368942
Epoch 14: Train err: 0.18725, Train loss: 0.39901529386639595 |Validation err: 0.31,
Validation loss: 0.7050973716974258
Epoch 15: Train err: 0.16525, Train loss: 0.36167532882094383 |Validation err: 0.324
5, Validation loss: 0.7471929438114167
Epoch 16: Train err: 0.157625, Train loss: 0.35454041412472725 |Validation err: 0.32
25, Validation loss: 0.7823102962970734
Epoch 17: Train err: 0.146, Train loss: 0.3308947429955006 |Validation err: 0.3125,
Validation loss: 0.7563041657209396
Epoch 18: Train err: 0.140125, Train loss: 0.31891468104720117 |Validation err: 0.31
8, Validation loss: 0.7705698120594024
Epoch 19: Train err: 0.133125, Train loss: 0.29830453781038524 |Validation err: 0.33
6, Validation loss: 0.998507696390152
Epoch 20: Train err: 0.122875, Train loss: 0.28220411322638395 |Validation err: 0.32
15, Validation loss: 0.9606681762933731
Epoch 21: Train err: 0.117, Train loss: 0.27377453028410675 |Validation err: 0.3245,
Validation loss: 0.9022090723514556
Epoch 22: Train err: 0.105125, Train loss: 0.24639589501544834 |Validation err: 0.32
85, Validation loss: 1.007312905550003
Epoch 23: Train err: 0.101, Train loss: 0.24810840704850853 |Validation err: 0.325,
Validation loss: 1.1917545728683472
Epoch 24: Train err: 0.100625, Train loss: 0.2458845970518887 |Validation err: 0.33
3, Validation loss: 1.151331667304039
Epoch 25: Train err: 0.095625, Train loss: 0.22709217967651785 |Validation err: 0.32
05, Validation loss: 1.1204603933095931
Epoch 26: Train err: 0.079375, Train loss: 0.19727520121634007 |Validation err: 0.32
7, Validation loss: 1.2134857638478278
Epoch 27: Train err: 0.0875, Train loss: 0.21331585030071437 |Validation err: 0.310
5, Validation loss: 1.2947224143743514
Epoch 28: Train err: 0.083875, Train loss: 0.2093619361743331 |Validation err: 0.325
5, Validation loss: 1.4272608540058136
```

```
Epoch 29: Train err: 0.08325, Train loss: 0.2018654831480235 |Validation err: 0.329
5, Validation loss: 1.4051471680402756
Epoch 30: Train err: 0.070875, Train loss: 0.17845182630605996 |Validation err: 0.3
2, Validation loss: 1.4173815153837204
Finished Training
Total time elapsed: 424.10 seconds
```



Decreasing the vatch size to 12 increased the training time to 424.10 seconds. This is because each epoch requires more batches and therefore, more parameter updates. With 8,000 training examples, a batch size of 16 requires approximately 500 batches per epoch, compared with approximately 125 batches when the batch size is 64.

The training error and training loss decrease consistently, reaching approximately 8% error and 0.18 loss. However, the validation loss begins increasing significantly after about epoch 8, eventually reaching approximately 1.4. The validation error also fluctuates and remains around 30-33% while the training error continues to fall.

This large gap between training and validation performance indicates strong overfitting. The smaller batch size produces noisier gradient estimates and many more updates. This can help the model fit the training data very closely, but in this experiment it reduces generalization to the validation data. Overall, decreasing the batch size made training slower and caused more noticeable overfitting.

Part 4. Hyperparameter Search [6 pt]

Part (a) - 2pt

Based on the plots from above, choose another set of values for the hyperparameters (network, batch_size, learning_rate) that you think would help you improve the validation

accuracy. Justify your choice.

- large_net with batch size of 64 and a learning rate of 0.005

Batch size 64 used since it was the most accurate compared to 512. But did not cause severe overfitting like 16. The learning rate of 0.1 caused unstable validation performance and overfitting, while 0.001 learned too slowly and still had relatively high validation error after 30 epochs. A learning rate of 0.005 is between these values and should provide more stable updates than 0.1 while learning faster than 0.001.

Part (b) - 1pt

Train the model with the hyperparameters you chose in part(a), and include the training curve.

In [ ]:
```
large_net = LargeNet()
train_net(large_net, batch_size=64, learning_rate=0.005, num_epochs=30)
plot_training_curve(get_model_name("large", batch_size=64, learning_rate=0.005, epo
```

```
Epoch 1: Train err: 0.498125, Train loss: 0.693441210269928 |Validation err: 0.463,
Validation loss: 0.692121647298336
Epoch 2: Train err: 0.449375, Train loss: 0.6902769436836242 |Validation err: 0.432
5, Validation loss: 0.6869329456239939
Epoch 3: Train err: 0.443625, Train loss: 0.6848875179290771 |Validation err: 0.423,
Validation loss: 0.6802606284618378
Epoch 4: Train err: 0.429125, Train loss: 0.6795436472892761 |Validation err: 0.420
5, Validation loss: 0.6714036520570517
Epoch 5: Train err: 0.41125, Train loss: 0.6712780375480651 |Validation err: 0.386,
Validation loss: 0.6615424510091543
Epoch 6: Train err: 0.380625, Train loss: 0.657154773235321 |Validation err: 0.3635,
Validation loss: 0.6462700515985489
Epoch 7: Train err: 0.373375, Train loss: 0.6469872641563416 |Validation err: 0.352,
Validation loss: 0.6381314136087894
Epoch 8: Train err: 0.365, Train loss: 0.6349571533203126 |Validation err: 0.3575, V
alidation loss: 0.6278157122433186
Epoch 9: Train err: 0.351375, Train loss: 0.6272215919494629 |Validation err: 0.346,
Validation loss: 0.6271738614886999
Epoch 10: Train err: 0.340375, Train loss: 0.6161677360534668 |Validation err: 0.34
1, Validation loss: 0.6169806346297264
Epoch 11: Train err: 0.331, Train loss: 0.6079148294925689 |Validation err: 0.3425,
Validation loss: 0.615937914699316
Epoch 12: Train err: 0.32225, Train loss: 0.5977423484325409 |Validation err: 0.342
5, Validation loss: 0.6228876728564501
Epoch 13: Train err: 0.317875, Train loss: 0.5934520001411439 |Validation err: 0.32
4, Validation loss: 0.6089944057166576
Epoch 14: Train err: 0.310625, Train loss: 0.5820841286182403 |Validation err: 0.34
4, Validation loss: 0.6233489066362381
Epoch 15: Train err: 0.299875, Train loss: 0.5745600435733795 |Validation err: 0.317
5, Validation loss: 0.5945671675726771
Epoch 16: Train err: 0.30425, Train loss: 0.5723140158653259 |Validation err: 0.334
5, Validation loss: 0.6094590704888105
Epoch 17: Train err: 0.291, Train loss: 0.5631587717533112 |Validation err: 0.3085,
Validation loss: 0.5952313458546996
Epoch 18: Train err: 0.28425, Train loss: 0.5524548044204712 |Validation err: 0.311
5, Validation loss: 0.5884092273190618
Epoch 19: Train err: 0.275, Train loss: 0.5422840743064881 |Validation err: 0.333, V
alidation loss: 0.6043308936059475
Epoch 20: Train err: 0.274625, Train loss: 0.5363975539207458 |Validation err: 0.31
2, Validation loss: 0.5995949814096093
Epoch 21: Train err: 0.270875, Train loss: 0.5346778197288513 |Validation err: 0.297
5, Validation loss: 0.5777320479974151
Epoch 22: Train err: 0.262375, Train loss: 0.5221112856864929 |Validation err: 0.30
7, Validation loss: 0.596011902205646
Epoch 23: Train err: 0.258625, Train loss: 0.5173619503974914 |Validation err: 0.290
5, Validation loss: 0.5722065921872854
Epoch 24: Train err: 0.252125, Train loss: 0.504859625339508 |Validation err: 0.304,
Validation loss: 0.5803852053359151
Epoch 25: Train err: 0.245, Train loss: 0.4957056167125702 |Validation err: 0.3065,
Validation loss: 0.5844235736876726
Epoch 26: Train err: 0.239625, Train loss: 0.4912502310276031 |Validation err: 0.30
8, Validation loss: 0.5841220626607537
Epoch 27: Train err: 0.240125, Train loss: 0.4875233781337738 |Validation err: 0.28
4, Validation loss: 0.5785381589084864
Epoch 28: Train err: 0.235875, Train loss: 0.4774924602508545 |Validation err: 0.293
5, Validation loss: 0.5843833107501268
```

```
Epoch 29: Train err: 0.22875, Train loss: 0.47385235714912416 |Validation err: 0.28
7, Validation loss: 0.5858967928215861
Epoch 30: Train err: 0.219125, Train loss: 0.4624751760959625 |Validation err: 0.282
5, Validation loss: 0.5806445851922035
Finished Training
Total time elapsed: 332.03 seconds
```

Part (c) - 2pt

Based on your result from Part(a), suggest another set of hyperparameter values to try. Justify your choice.

- increase batch size to 128, keep a learning rate of 0.005

Batch size of 512 trained faster, but had a weaker validation performance, while batch size of 16 had substanisial overfitting. Currently, there seems to be some overfitting in teh graphs which could be reduced by increasing the batch size.

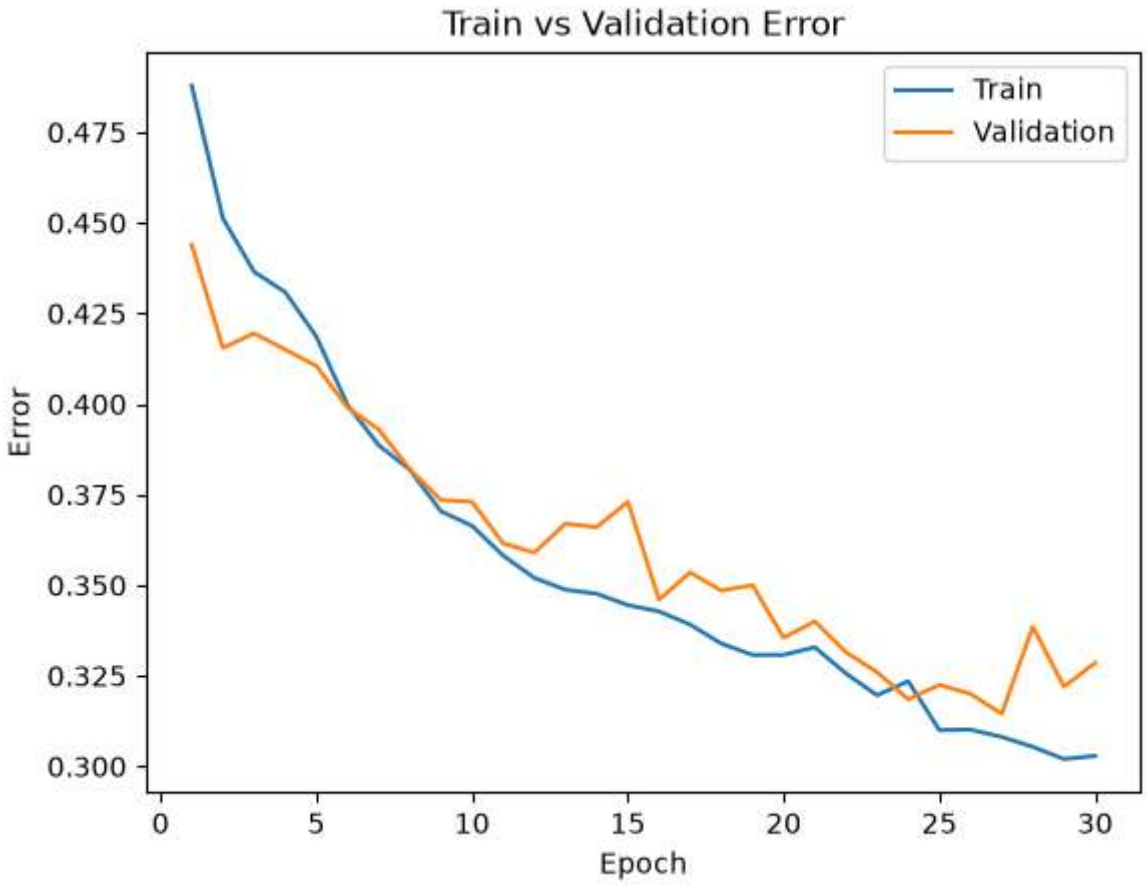
Part (d) - 1pt

Train the model with the hyperparameters you chose in part(c), and include the training curve.

In [ ]:
```
large_net = LargeNet()
train_net(large_net, batch_size=128, learning_rate=0.005, num_epochs=30)
plot_training_curve(get_model_name("large", batch_size=128, learning_rate=0.005, ep
```

```
Epoch 1: Train err: 0.487875, Train loss: 0.693385036218734 |Validation err: 0.444,
Validation loss: 0.6909762062132359
Epoch 2: Train err: 0.45125, Train loss: 0.6888883634219094 |Validation err: 0.4155,
Validation loss: 0.685368612408638
Epoch 3: Train err: 0.4365, Train loss: 0.6843413776821561 |Validation err: 0.4195,
Validation loss: 0.6805739812552929
Epoch 4: Train err: 0.430875, Train loss: 0.6794616002885122 |Validation err: 0.415,
Validation loss: 0.6749377213418484
Epoch 5: Train err: 0.418625, Train loss: 0.6742368917616587 |Validation err: 0.410
5, Validation loss: 0.6712135635316372
Epoch 6: Train err: 0.39975, Train loss: 0.668240279432327 |Validation err: 0.399, V
alidation loss: 0.6664445698261261
Epoch 7: Train err: 0.388625, Train loss: 0.6615137741679237 |Validation err: 0.393,
Validation loss: 0.6584289744496346
Epoch 8: Train err: 0.381875, Train loss: 0.6546839295871674 |Validation err: 0.382,
Validation loss: 0.6526732593774796
Epoch 9: Train err: 0.370375, Train loss: 0.6473599615551177 |Validation err: 0.373
5, Validation loss: 0.64577791467309
Epoch 10: Train err: 0.366375, Train loss: 0.644250992744688 |Validation err: 0.373,
Validation loss: 0.6441583447158337
Epoch 11: Train err: 0.358125, Train loss: 0.6377602001977345 |Validation err: 0.361
5, Validation loss: 0.6410304866731167
Epoch 12: Train err: 0.352, Train loss: 0.6310294062372238 |Validation err: 0.359, V
alidation loss: 0.6350990980863571
Epoch 13: Train err: 0.34875, Train loss: 0.628263238876585 |Validation err: 0.367,
Validation loss: 0.6348069533705711
Epoch 14: Train err: 0.347625, Train loss: 0.623179400724078 |Validation err: 0.366,
Validation loss: 0.6376562379300594
Epoch 15: Train err: 0.3445, Train loss: 0.6199877110738603 |Validation err: 0.373,
Validation loss: 0.6487049013376236
Epoch 16: Train err: 0.34275, Train loss: 0.6174203195269146 |Validation err: 0.346,
Validation loss: 0.6304149925708771
Epoch 17: Train err: 0.339125, Train loss: 0.6172576839961703 |Validation err: 0.353
5, Validation loss: 0.6268146336078644
Epoch 18: Train err: 0.333875, Train loss: 0.6108771609881568 |Validation err: 0.348
5, Validation loss: 0.6218162290751934
Epoch 19: Train err: 0.33075, Train loss: 0.6079039762890528 |Validation err: 0.35,
Validation loss: 0.6251150257885456
Epoch 20: Train err: 0.33075, Train loss: 0.6062725167425852 |Validation err: 0.335
5, Validation loss: 0.6182570531964302
Epoch 21: Train err: 0.332875, Train loss: 0.6029126927966163 |Validation err: 0.34,
Validation loss: 0.6181245855987072
Epoch 22: Train err: 0.325625, Train loss: 0.5979667618161156 |Validation err: 0.331
5, Validation loss: 0.6184801533818245
Epoch 23: Train err: 0.319625, Train loss: 0.5955922594146122 |Validation err: 0.32
6, Validation loss: 0.6120428778231144
Epoch 24: Train err: 0.3235, Train loss: 0.5945297678311666 |Validation err: 0.3185,
Validation loss: 0.6082612723112106
Epoch 25: Train err: 0.31, Train loss: 0.5850105531631954 |Validation err: 0.3225, V
alidation loss: 0.6074993945658207
Epoch 26: Train err: 0.310125, Train loss: 0.5812255655016217 |Validation err: 0.32,
Validation loss: 0.6028802879154682
Epoch 27: Train err: 0.308125, Train loss: 0.5787285291959369 |Validation err: 0.314
5, Validation loss: 0.6046864613890648
Epoch 28: Train err: 0.305375, Train loss: 0.574344881943294 |Validation err: 0.338
5, Validation loss: 0.6164047420024872
```

```
Epoch 29: Train err: 0.302, Train loss: 0.5747079768824199 |Validation err: 0.322, V
alidation loss: 0.6126517280936241
Epoch 30: Train err: 0.302875, Train loss: 0.5727003916861519 |Validation err: 0.328
5, Validation loss: 0.6096509620547295
Finished Training
Total time elapsed: 308.37 seconds
```



Part 4. Evaluating the Best Model [15 pt]

Part (a) - 1pt

Choose the **best** model that you have so far. This means choosing the best model checkpoint, including the choice of `small_net` vs `large_net`, the `batch_size`, `learning_rate`, **and the epoch number.**

Modify the code below to load your chosen set of weights to the model object `net`.

In [ ]:
```
net = LargeNet()
model_path = get_model_name(net.name, batch_size=64, learning_rate=0.005, epoch=29)
state = torch.load(model_path)
net.load_state_dict(state)
```

Out[ ]:
```
<All keys matched successfully>
```

Part (b) - 2pt

Justify your choice of model from part (a).

The model does show moderate overfitting because its training error was 0.2191 compared with a validation error of 0.2825, but its validation performance was still the best overall.

Therefore, this checkpoint provides the best balance between accuracy and generalization among all the experiments.

Large Net:

- achieved a lower validation error than small_net, indicating better performance on unseen validation images

Batch size = 64:

- because smaller batches, especially 16, produced substantially more overfitting, while the larger batch size of 512 had worse validation performance

Learning Rate = 0.005:

- learning rate of 0.001 learned too slowly, while 0.01 and 0.1 resulted in greater instability or overfitting

Epoch-30 checkpoint:

- it had the lowest validation classification error among the checkpoints tested, approximately 0.2825

Part (c) - 2pt

Using the code in Part 0, any code from lecture notes, or any code that you write, compute and report the **test classification error** for your chosen model.

In [ ]:
```
# If you use the `evaluate` function provided in part 0, you will need to
# set batch_size > 1
train_loader, val_loader, test_loader, classes = get_data_loader(
    target_classes=["cat", "dog"],
    batch_size=64)

# Use the same loss used during training
criterion = nn.BCEWithLogitsLoss()

# Evaluate the model on the test set
test_err, test_loss = evaluate(net, test_loader, criterion)
# from evaluate function - classification rule
#corr = (outputs > 0.0).squeeze().long() != labels

print("Test classification error:", test_err)
print("Test classification accuracy:", 1 - test_err)
```

```
Test classification error: 0.2985
Test classification accuracy: 0.7015
```

Part (d) - 3pt

How does the test classification error compare with the **validation error**? Explain why you would expect the test error to be *higher* than the validation error.

The test classification error is 0.2985, while the validation error of the selected model was approximately 0.2825. Therefore, the test error is higher by: $0.2985 - 0.2825 = 0.016$.

The test error is higher because the validation set was used during model selection and hyperparameter tuning. We used validation performance to choose the network, batch size, learning rate, and checkpoint epoch, so the model selection process was indirectly optimized for the validation set. The test set was kept separate and was not used during training or tuning, so it provides a more unbiased estimate of how the model performs on unseen data. The difference can also partly result from natural variation between the validation and test examples.

Part (e) - 2pt

Why did we only use the test data set at the very end? Why is it important that we use the test data as little as possible?

We used the test dataset only at the end because it provides an unbiased estimate of how the final model performs on completely unseen data. During training and hyperparameter tuning, we used the training and validation sets instead.

It is important to use the test data as little as possible because repeatedly checking test performance can influence our decisions and cause the model to overfit to the test set. If that happens, the test error will no longer accurately represent performance on genuinely new data.

Part (f) - 5pt

How does the your best CNN model compare with an 2-layer ANN model (no convolutional layers) on classifying cat and dog images. You can use a 2-layer ANN architecture similar to what you used in Lab 1. You should explore different hyperparameter settings to determine how well you can do on the validation dataset. Once satisified with the performance, you may test it out on the test data.

Hint: The ANN in lab 1 was applied on greyscale images. The cat and dog images are colour (RGB) and so you will need to flatted and concatinate all three colour layers before feeding them into an ANN.

In [16]:
```
# define the ANN
class ANN(nn.Module):
    def __init__(self):
        super(ANN, self).__init__()
        self.name = "ann"
        self.layer1 = nn.Linear(3 * 32 * 32, 30)
```

```
        self.layer2 = nn.Linear(30, 1)

    def forward(self, image):
        flattened = image.reshape(-1, 3 * 32 * 32)
        activation1 = F.relu(self.layer1(flattened))
        output = self.layer2(activation1)
        return output.squeeze(1)

#convert the labels for cat (3) and dog (5) to binary labels for the ANN
def ann_label(labels):
    return (labels == 5).float()

def evaluate_ann(net, loader, criterion):
    total_error = 0
    total_loss = 0
    total_examples = 0

    net.eval()
    with torch.no_grad():
        for inputs, labels in loader:
            labels = ann_label(labels)
            outputs = net(inputs)

            loss = criterion(outputs, labels)
            predictions = (outputs > 0).long()
            total_error += int((predictions != labels.long()).sum())
            total_loss += loss.item() * len(labels)
            total_examples += len(labels)

    return total_error / total_examples, total_loss / total_examples
```

In [ ]:

```
ann = ANN()
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(ann.parameters(), lr=0.0005, momentum=0.9)

train_errors = []
val_errors = []
train_losses = []
val_losses = []

for iteration in range(30):
    ann.train()

    for images, labels in train_loader:
        labels = ann_label(labels)

        optimizer.zero_grad()
        outputs = ann(images)
        loss = criterion(outputs, labels)
        loss.backward()

        optimizer.step()

    train_error, train_loss = evaluate_ann(ann, train_loader, criterion)
    val_error, val_loss = evaluate_ann(ann, val_loader, criterion)
```

```
    train_errors.append(train_error)
    val_errors.append(val_error)
    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(
        f"Iteration {iteration + 1}: "
        f"Train error = {train_error:.4f}, "
        f"Validation error = {val_error:.4f}"
    )
```

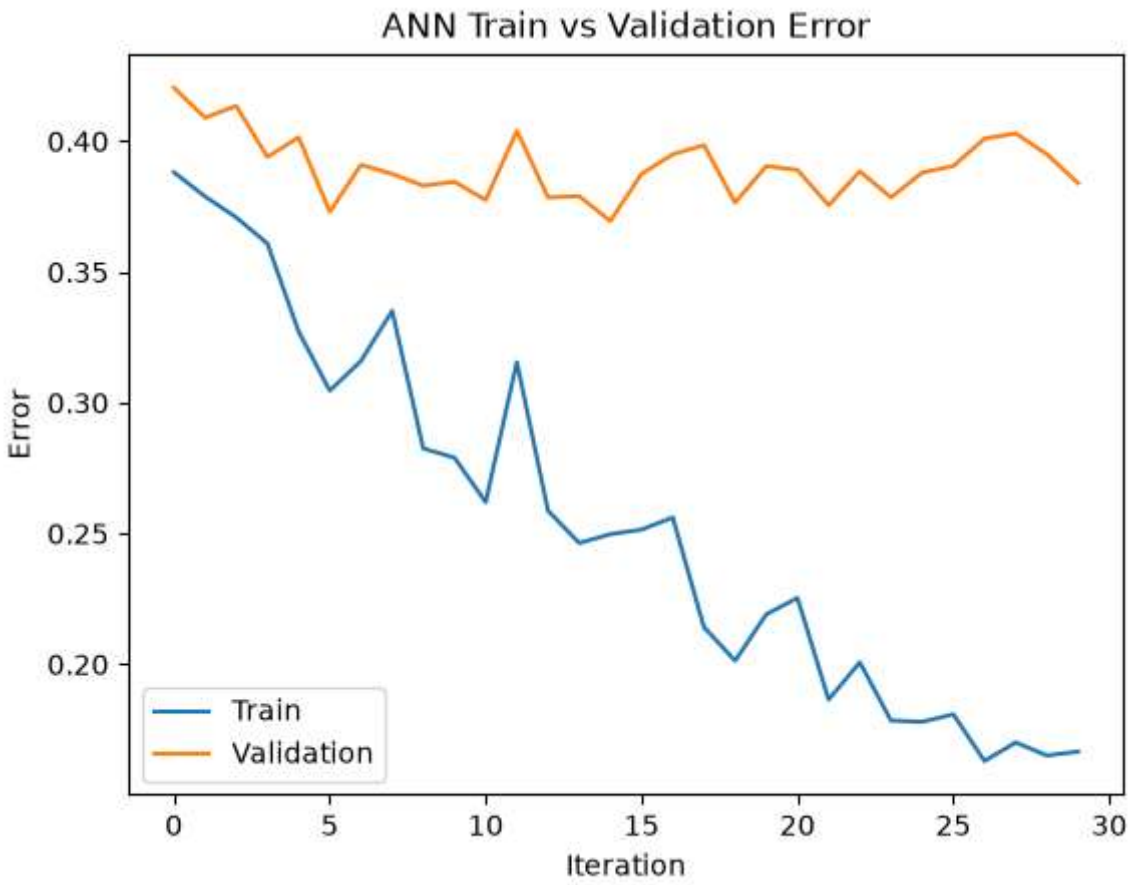
```
Iteration 1: Train error = 0.3881, Validation error = 0.4205
Iteration 2: Train error = 0.3789, Validation error = 0.4090
Iteration 3: Train error = 0.3709, Validation error = 0.4135
Iteration 4: Train error = 0.3609, Validation error = 0.3940
Iteration 5: Train error = 0.3272, Validation error = 0.4015
Iteration 6: Train error = 0.3046, Validation error = 0.3730
Iteration 7: Train error = 0.3160, Validation error = 0.3910
Iteration 8: Train error = 0.3351, Validation error = 0.3875
Iteration 9: Train error = 0.2825, Validation error = 0.3830
Iteration 10: Train error = 0.2790, Validation error = 0.3845
Iteration 11: Train error = 0.2620, Validation error = 0.3775
Iteration 12: Train error = 0.3155, Validation error = 0.4040
Iteration 13: Train error = 0.2587, Validation error = 0.3785
Iteration 14: Train error = 0.2464, Validation error = 0.3790
Iteration 15: Train error = 0.2497, Validation error = 0.3695
Iteration 16: Train error = 0.2515, Validation error = 0.3875
Iteration 17: Train error = 0.2561, Validation error = 0.3950
Iteration 18: Train error = 0.2142, Validation error = 0.3985
Iteration 19: Train error = 0.2014, Validation error = 0.3765
Iteration 20: Train error = 0.2191, Validation error = 0.3905
Iteration 21: Train error = 0.2254, Validation error = 0.3890
Iteration 22: Train error = 0.1865, Validation error = 0.3755
Iteration 23: Train error = 0.2008, Validation error = 0.3885
Iteration 24: Train error = 0.1785, Validation error = 0.3785
Iteration 25: Train error = 0.1780, Validation error = 0.3880
Iteration 26: Train error = 0.1809, Validation error = 0.3905
Iteration 27: Train error = 0.1630, Validation error = 0.4010
Iteration 28: Train error = 0.1701, Validation error = 0.4030
Iteration 29: Train error = 0.1651, Validation error = 0.3950
Iteration 30: Train error = 0.1666, Validation error = 0.3840
```

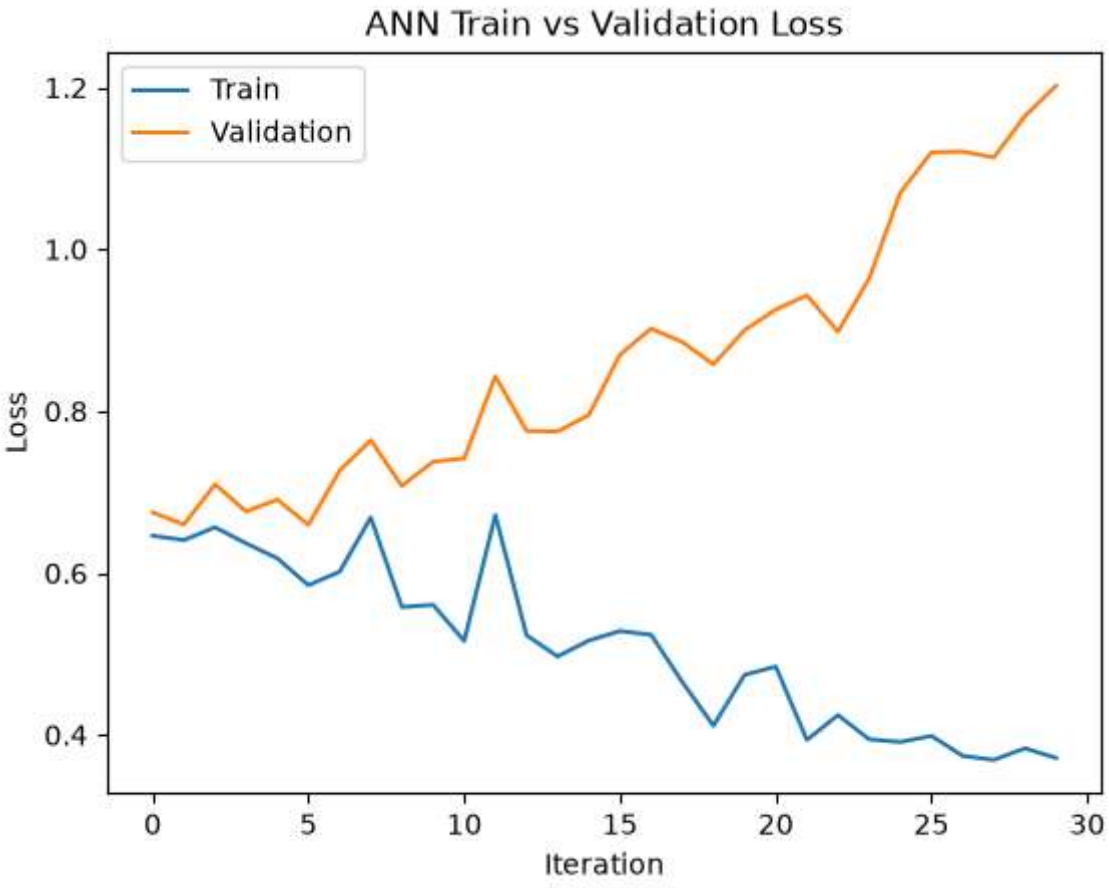
In [18]:

```
plt.plot(train_errors, label="Train")
plt.plot(val_errors, label="Validation")
plt.xlabel("Iteration")
plt.ylabel("Error")
plt.title("ANN Train vs Validation Error")
plt.legend()
plt.show()

plt.plot(train_losses, label="Train")
plt.plot(val_losses, label="Validation")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("ANN Train vs Validation Loss")
```

```
plt.legend()
plt.show()
```





In [19]:
```
ann_test_error, ann_test_loss = evaluate_ann(
    ann,
    test_loader,
    criterion
)

print("ANN test classification error:", ann_test_error)
print("ANN test classification accuracy:", 1 - ann_test_error)
```

```
ANN test classification error: 0.3985
ANN test classification accuracy: 0.6014999999999999
```

I compared the best CNN, a LargeNet, with a two-layer ANN based on the architecture used in Lab 1. Since CIFAR-10 images are RGB, each image was flattened from 3×32×32 into 3,072 input features. The ANN used one hidden layer with 30 units and ReLU activation.

The best ANN achieved a validation error of approximately 0.364, corresponding to a validation accuracy of 63.6%. Its test error was 0.3545, corresponding to a test accuracy of 64.55%.

The CNN achieved a validation error of approximately 0.2825, or 71.75% validation accuracy. Its test error was 0.2985, or 70.15% test accuracy.

Therefore, the CNN performed better than the ANN:

Validation accuracy: CNN 71.75% vs. ANN 63.6% Test accuracy: CNN 70.15% vs. ANN 64.55% The CNN was better because convolutional layers preserve the spatial relationships between neighboring pixels and can learn local image features such as edges and textures. The ANN flattened the entire image before processing it, so it lost this spatial information. Although the ANN training error decreased substantially, its validation error remained higher, indicating some overfitting and weaker generalization. Overall, the CNN was more effective for classifying the cat and dog images.